AI IN ACTION · NGÀY 5

Thiết kế sản phẩm AI cho sự không chắc chắn

Từ khả năng của model đến trải nghiệm đáng tin cậy của người dùng

Instructor: Mai Anh Nguyen (Blue) · VinUniversity · Day 5 · 2026

Instructor

Mai Anh Nguyen (Blue)

Generalist Product Builder

- 2026 FPT Long Châu (PM · Healthcare Product)
- 2025 Thongtincuuho.org (Co-founder)
- 2025 FPT Software AI Center (PM · AI Agent)
- 2021 - 2025 Xantus (PM · On-chain Analytics, AI Agent)
- 2016 - 2021 DYNO, Kalapa (PM · OCR, eKYC, Credit Scoring)

Linkedin | Facebook

Agenda

- AI Prototyping & MVP
- Scope & PRD cho AI feature
- Human-centered AI design
- Evals flow

Thiết kế sản phẩm AI cho sự không chắc chắn

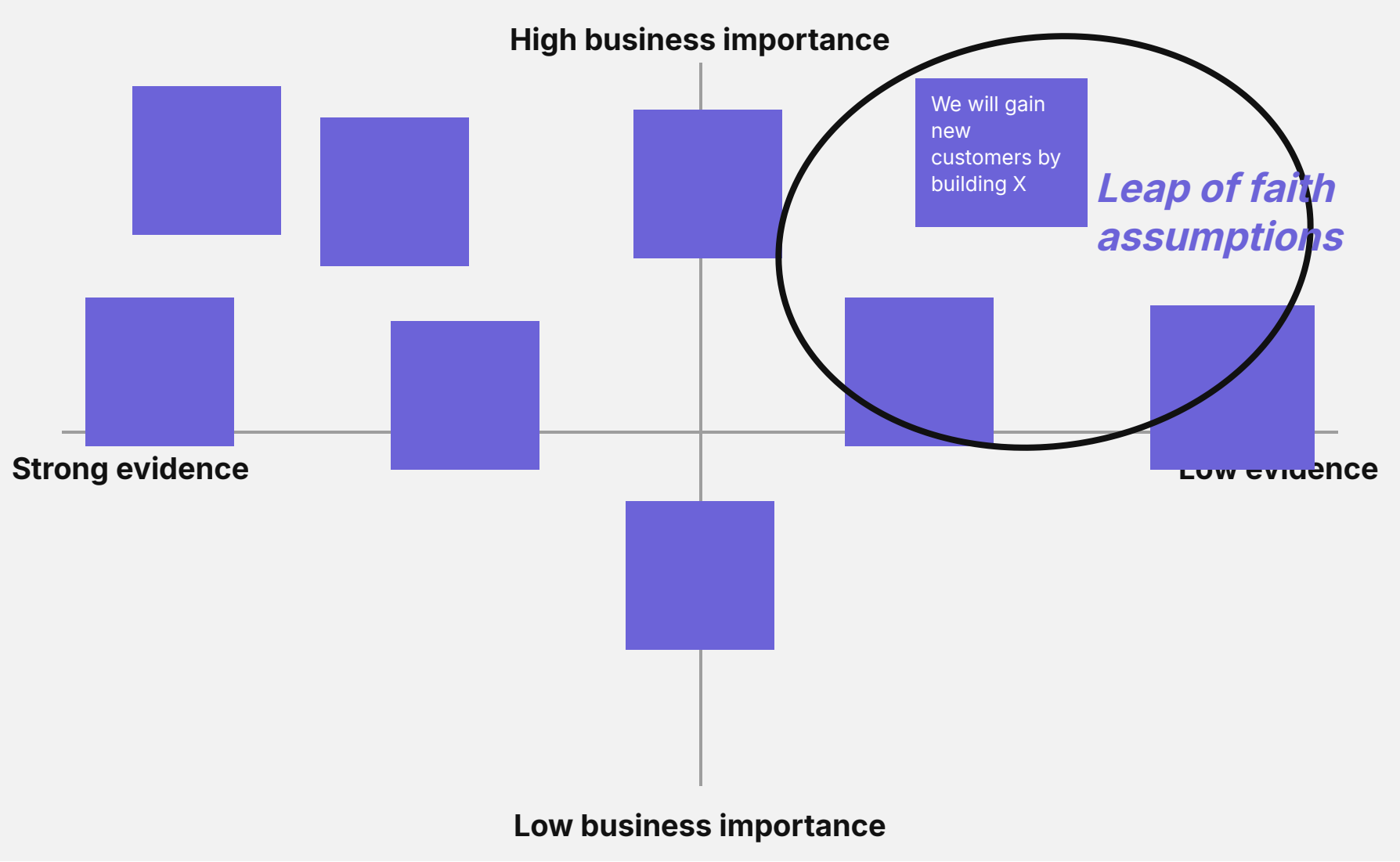
Từ prototype rẻ nhất → spec đúng → đo chất lượng bằng chu trình

SECTION 01

AI Prototyping & MVP

Test giả thuyết rẻ nhất trước khi build

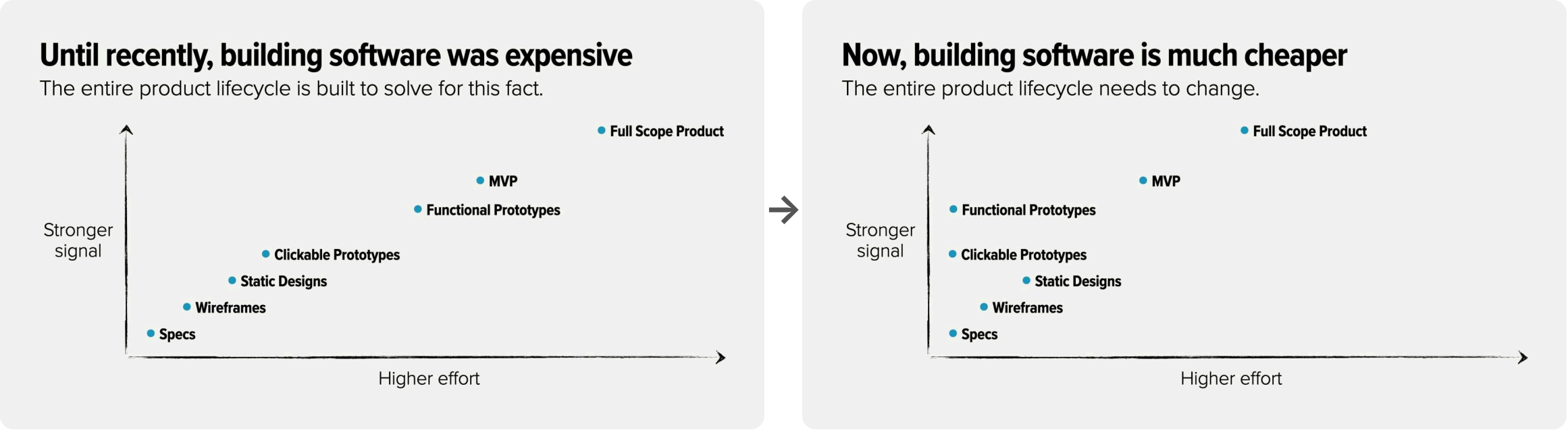
Leap of Faith Assumptions (The Lean Startup)



- Will the customer buy this, or choose to use it? (Value risk)
  Khách có mua — hoặc chọn dùng — không?
- Can the user figure out how to use it? (Usability risk)
  User có tự biết cách dùng không?
- Can we build it? (Feasibility risk)
  Ta có build được không?
- Does this solution work for our business? (Business viability risk)
  Giải pháp này có hợp với business của ta không?

Build software: từ rất đắt → rẻ hơn nhiều

Toàn bộ product lifecycle được thiết kế quanh việc "build là đắt" — khi build rẻ đi, lifecycle đó phải đổi theo.

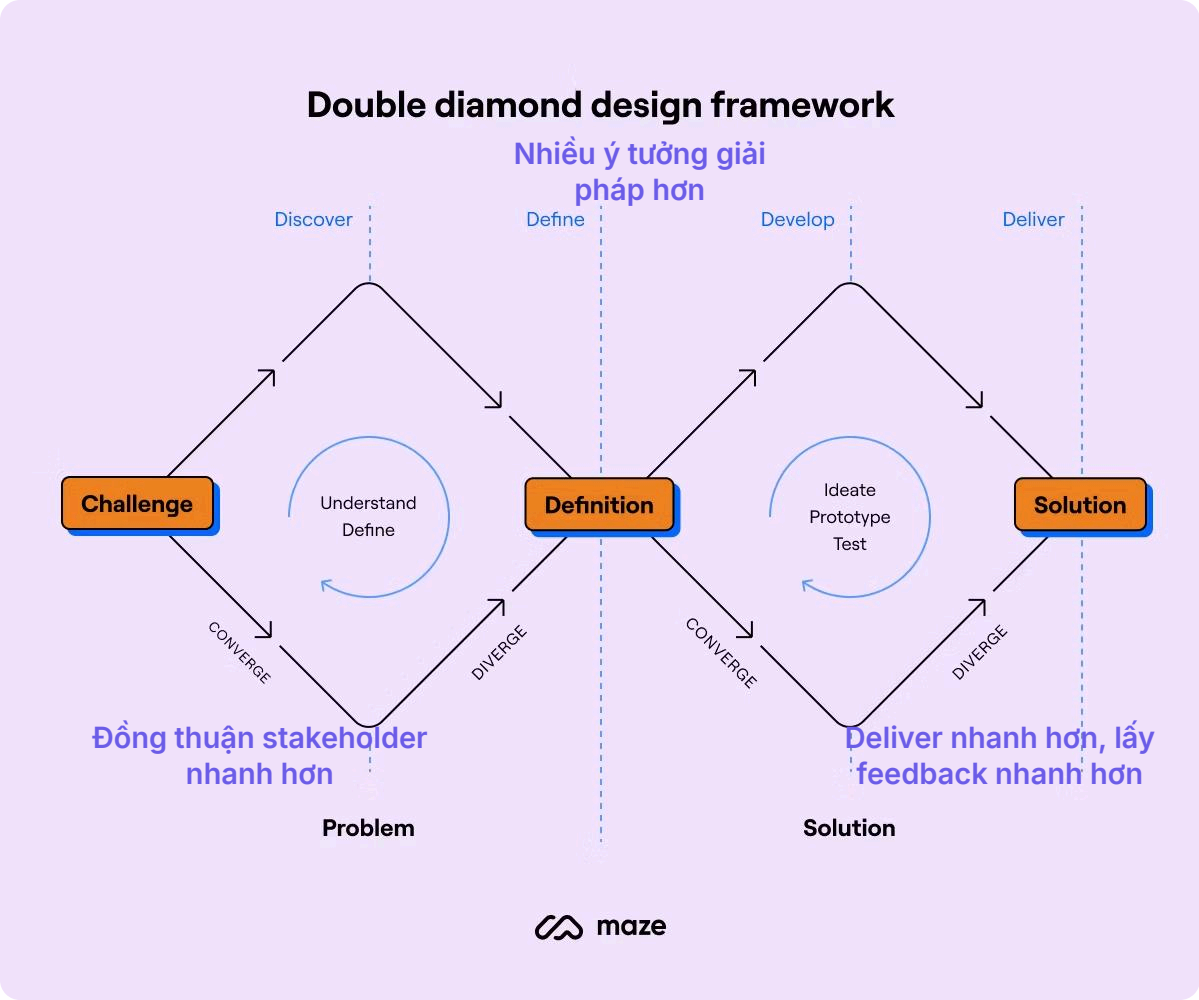


Cùng một thang "signal vs effort" — MVP và prototype giờ nằm ở chỗ trước đây chỉ có wireframe.

Nguồn: Day 17 lecture — "Until recently, building software was expensive" / "Now, building software is much cheaper".

Với AI, cả 3 khâu đều nhanh hơn

Double diamond không đổi — tốc độ đi qua nó thay đổi hoàn toàn.



Prototype as decision-making tools — for exploration, alignment, and validation.

"If you aren't prototyping with AI, you're doing it wrong" — Microsoft CPO.

Thang fidelity: Sketch → Wireframe → Mockup → Prototype

Low-fidelity kiểm tra luồng & tính năng — high-fidelity kiểm tra trải nghiệm "như thật".

LOW-FIDELITY

HIGH-FIDELITY

Đúng luồng / workflow chưa?

Người dùng có thấy rõ tính năng không?

Mô phỏng trông như thật

Test với người dùng cuối

01
Sketch

02
Wireframe
functional · structure

03
Mockup
style · color

04
Prototype
interactive · clickable

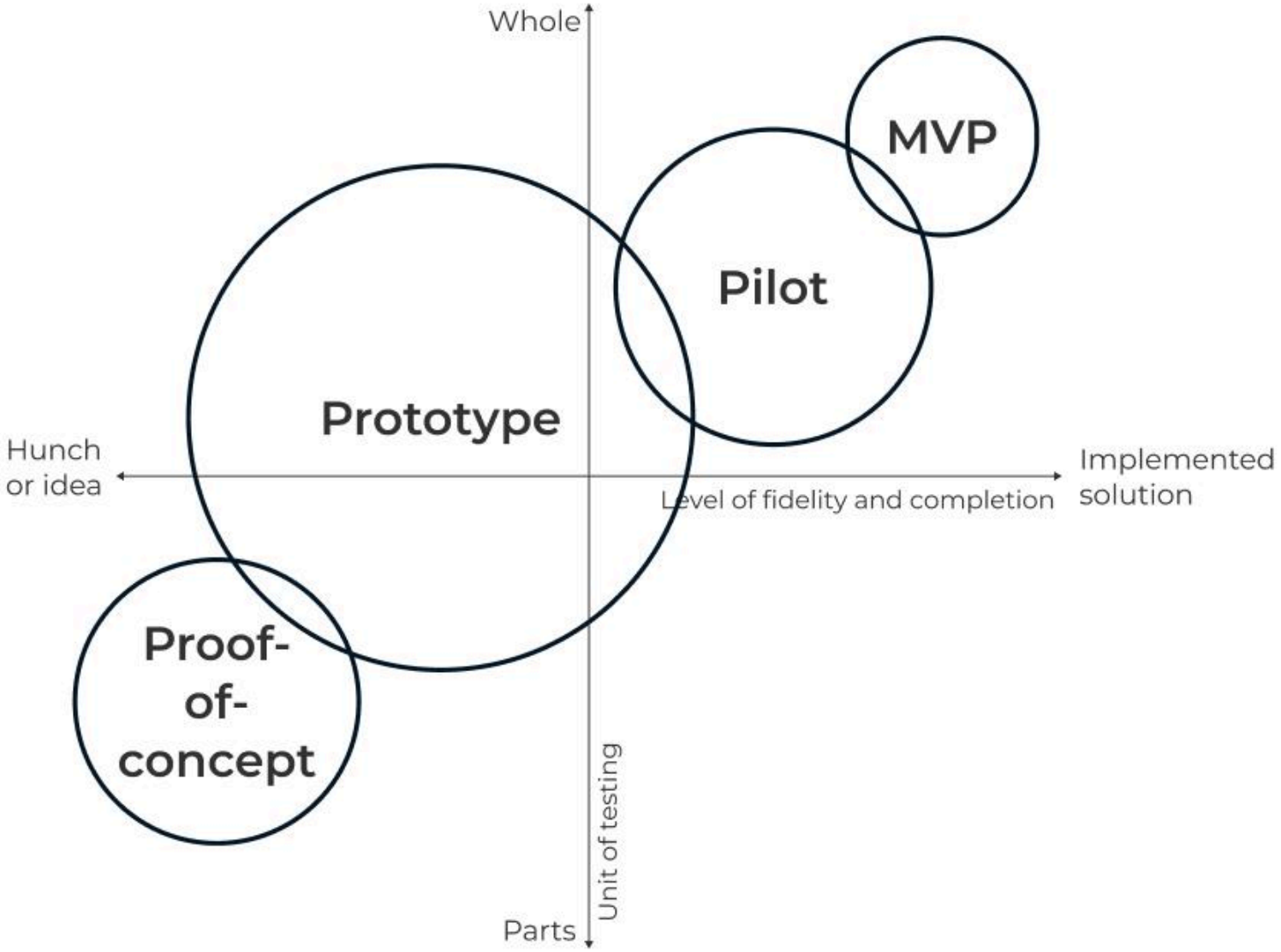
ĐỘ TRUNG THỰC (FIDELITY) TĂNG DẦN

Low-fi chốt luồng & tính năng — high-fi kiểm chứng "trông như thật" với người dùng cuối.

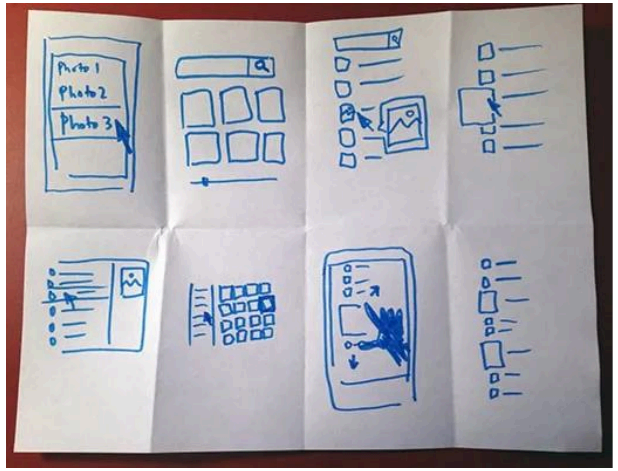
Sketch → Wireframe → Mockup → Prototype: fidelity và effort tăng dần theo từng bậc.

Prototype · Pilot · MVP · Proof-of-concept — định nghĩa cho rõ

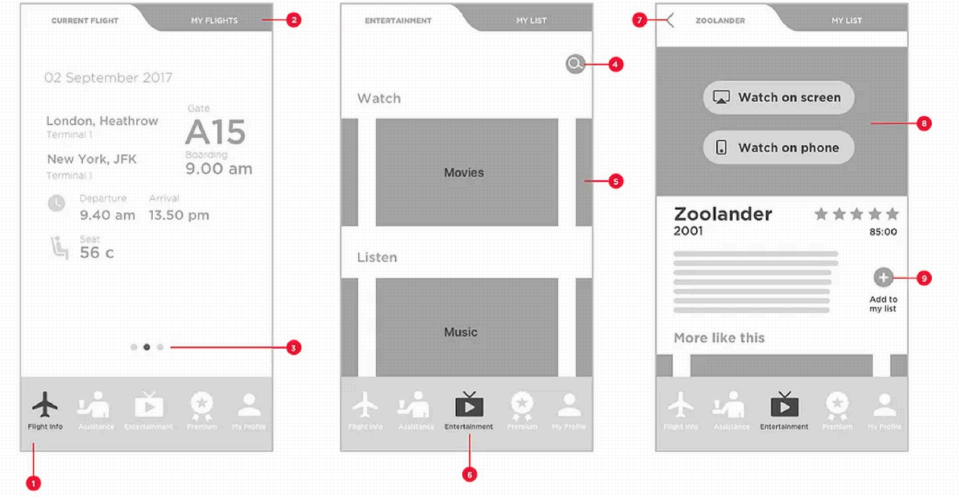
Trục ngang: mức độ hoàn thiện (fidelity & completion) · Trục dọc: phạm vi test (parts → whole).



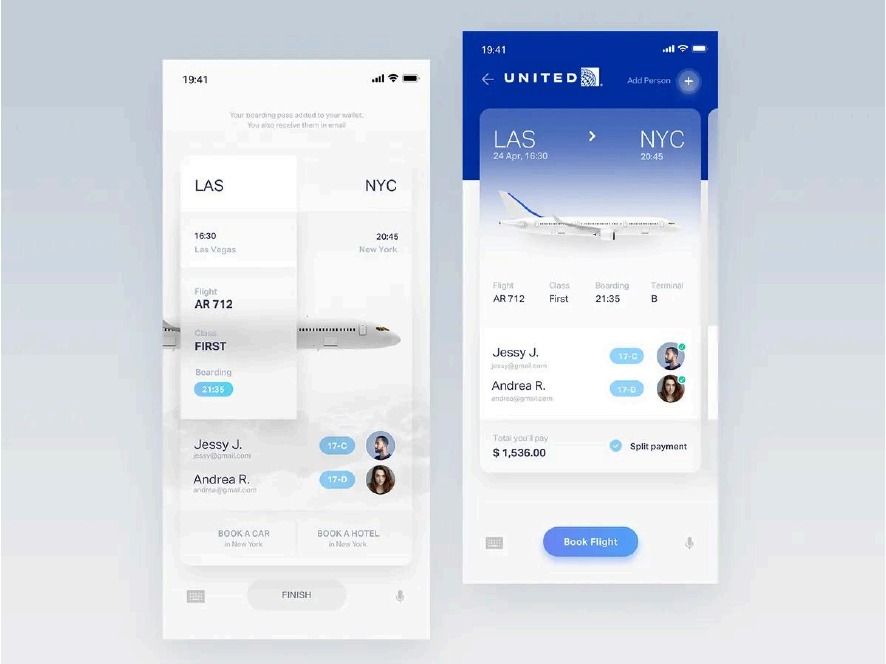
Sketch



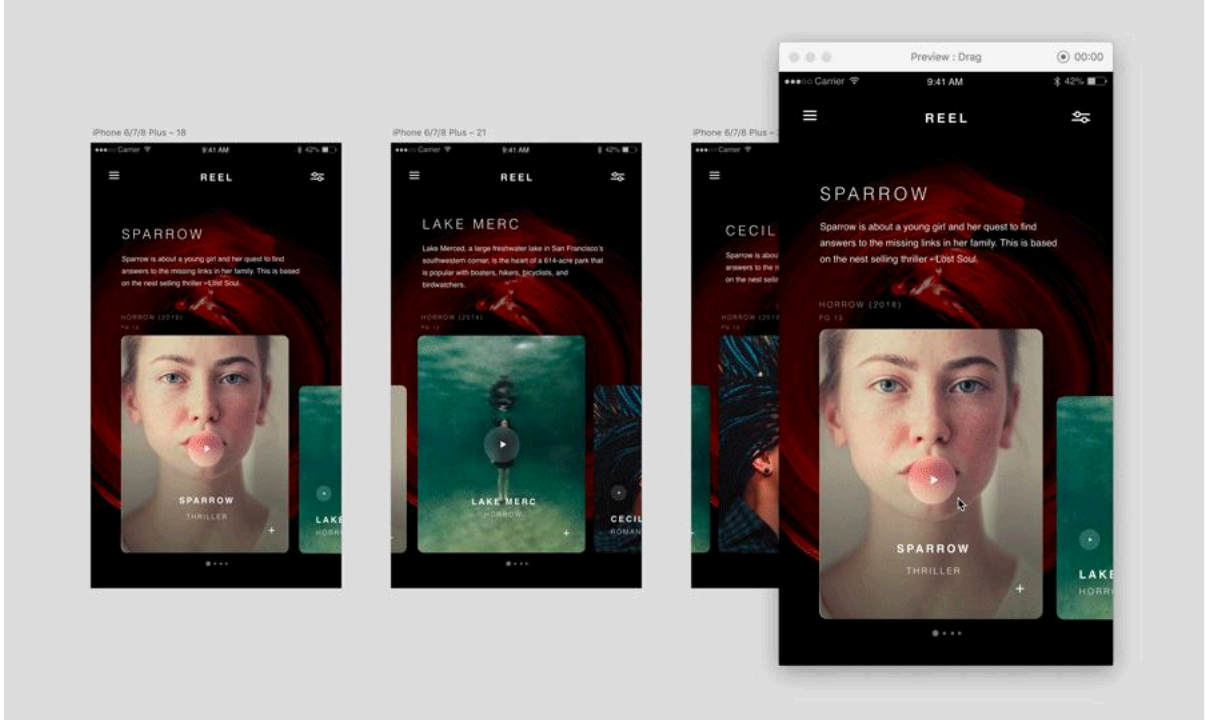
Wireframe functional, structure



Mockup style, color



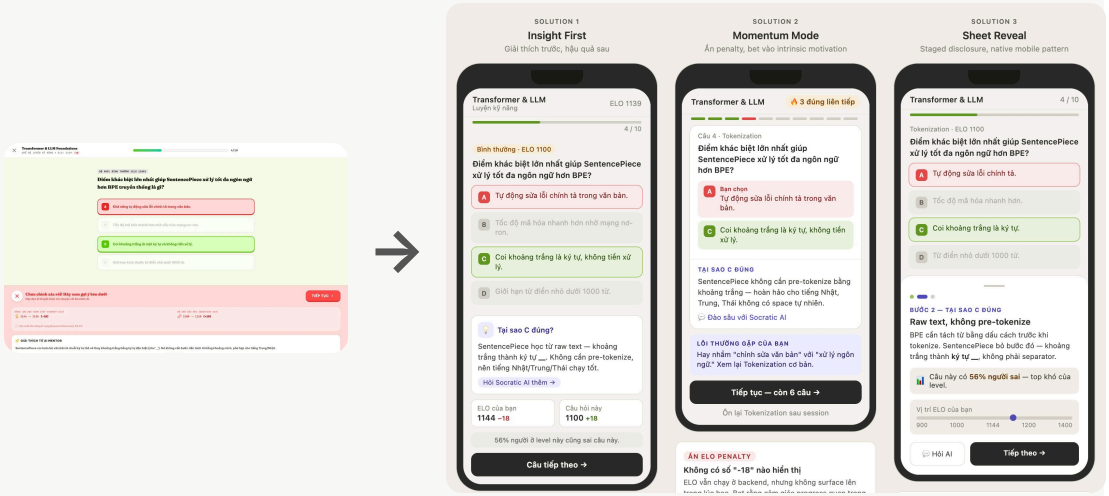
Prototype Interactive, clickable



MVP: cách nào rẻ nhất để test được giả thuyết?

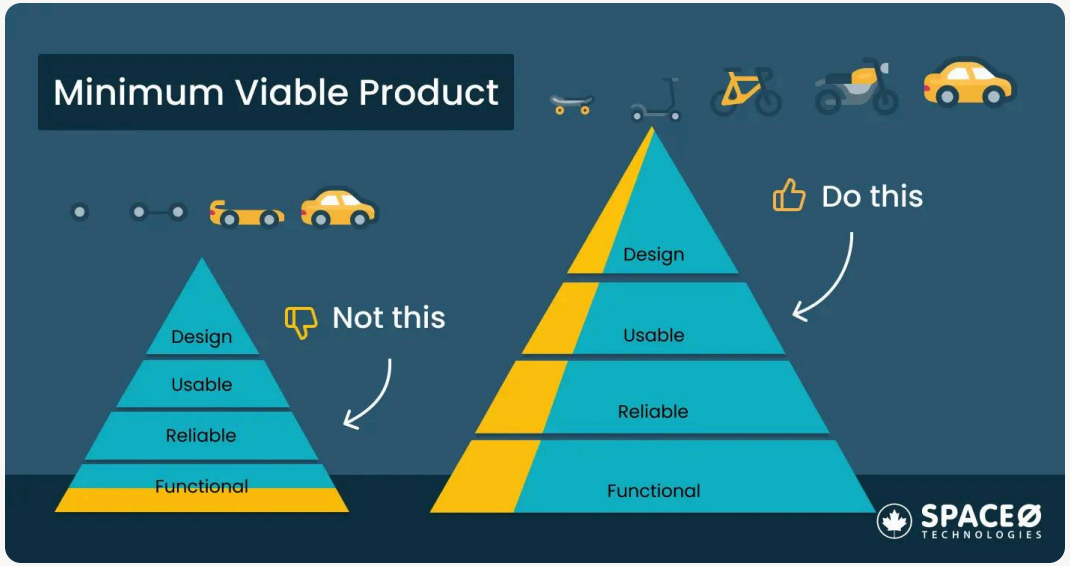
Using **variations** for brainstorming — nhiều biến thể, rẻ, trước khi chọn một.

① 1 màn hình → 3 biến thể trong vài giờ



Trước: 1 màn hình desktop duy nhất → Sau: Insight First · Momentum Mode · Sheet Reveal — cùng một bài toán, 3 hướng giải, so sánh trước khi cược vào một.

② Lát cắt mỏng xuyên suốt, không phải một tầng hoàn chỉnh



Không build "bánh xe trước, xe hơi sau". Build chiếc xe đạp chạy được ngay — nhỏ nhưng đủ cả 4 tầng.

The old way

PM → Design → Engineering

Weeks of lag

Rigid handoffs

Limited customer engagement

The new model

PM + Design + Engineering

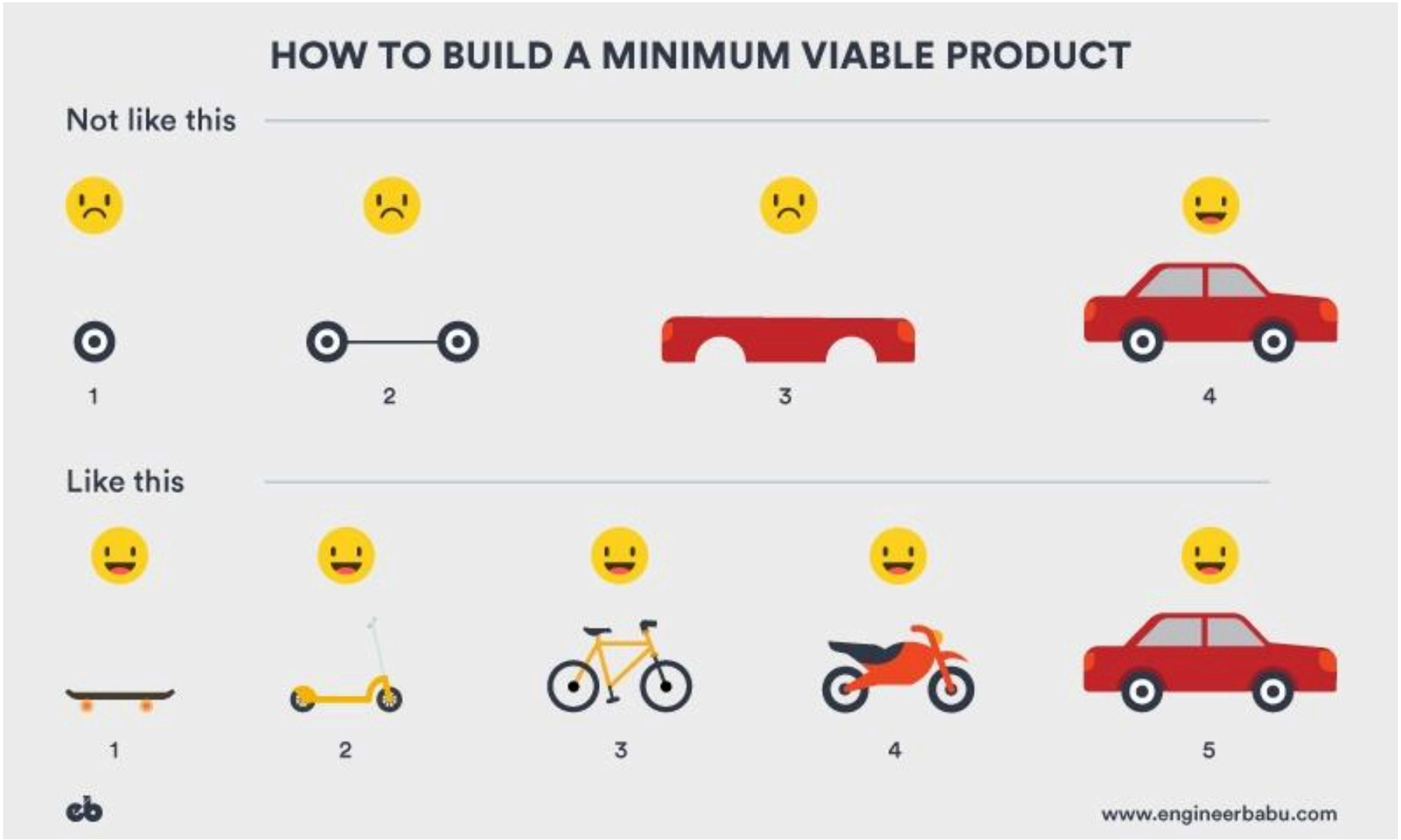
Build in hours

Collaborative riffing

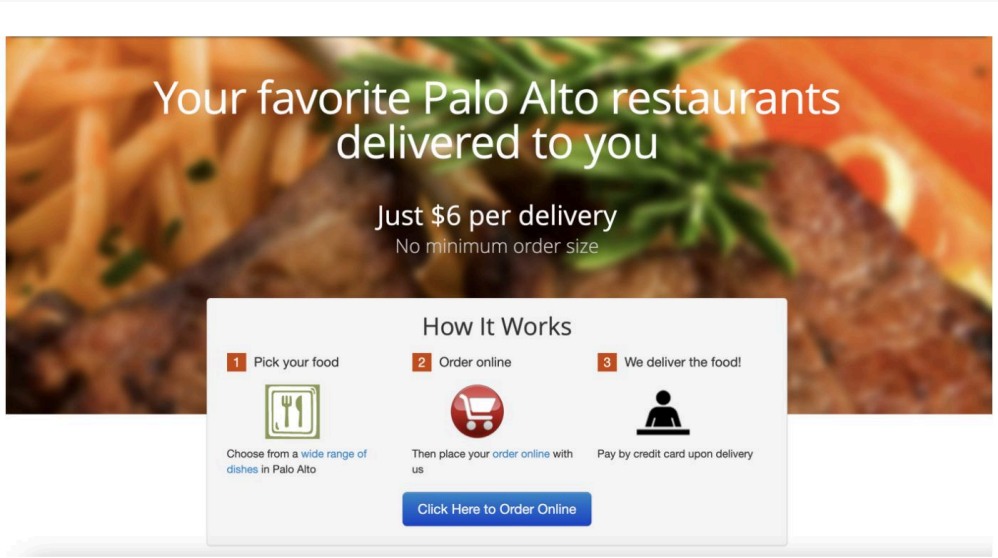
More customer conversations

MVP: build cái dùng được, không build từng mảnh

Not like this: từng bộ phận rời — user chưa dùng được gì. Like this: ván trượt → xe đạp → xe máy → ô tô, mỗi bước đều đi được.



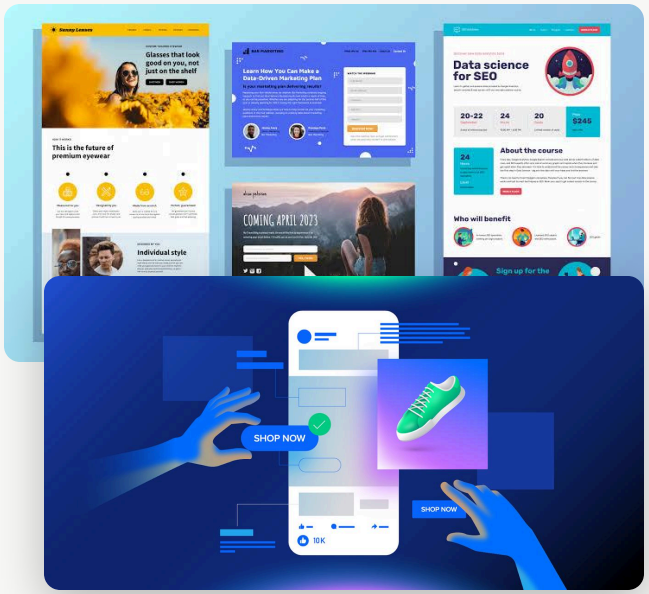
Wizard of Oz MVP



DoorDash - "Palo Alto Delivery" (2013)

- Giả định: "Có ai cần giao đồ ăn từ quán địa phương không?"
- MVP: một trang web tĩnh + PDF menu 8 quán (không xin phép), để một số Google Voice chung (đổ chuông máy cả 4 founder). Có cuộc gọi → tự gọi đặt món, tự lái đi giao, lấy $6. Dán tờ rơi quanh Stanford — đơn đầu tiên từ khách lạ qua Google sau ~45 phút.

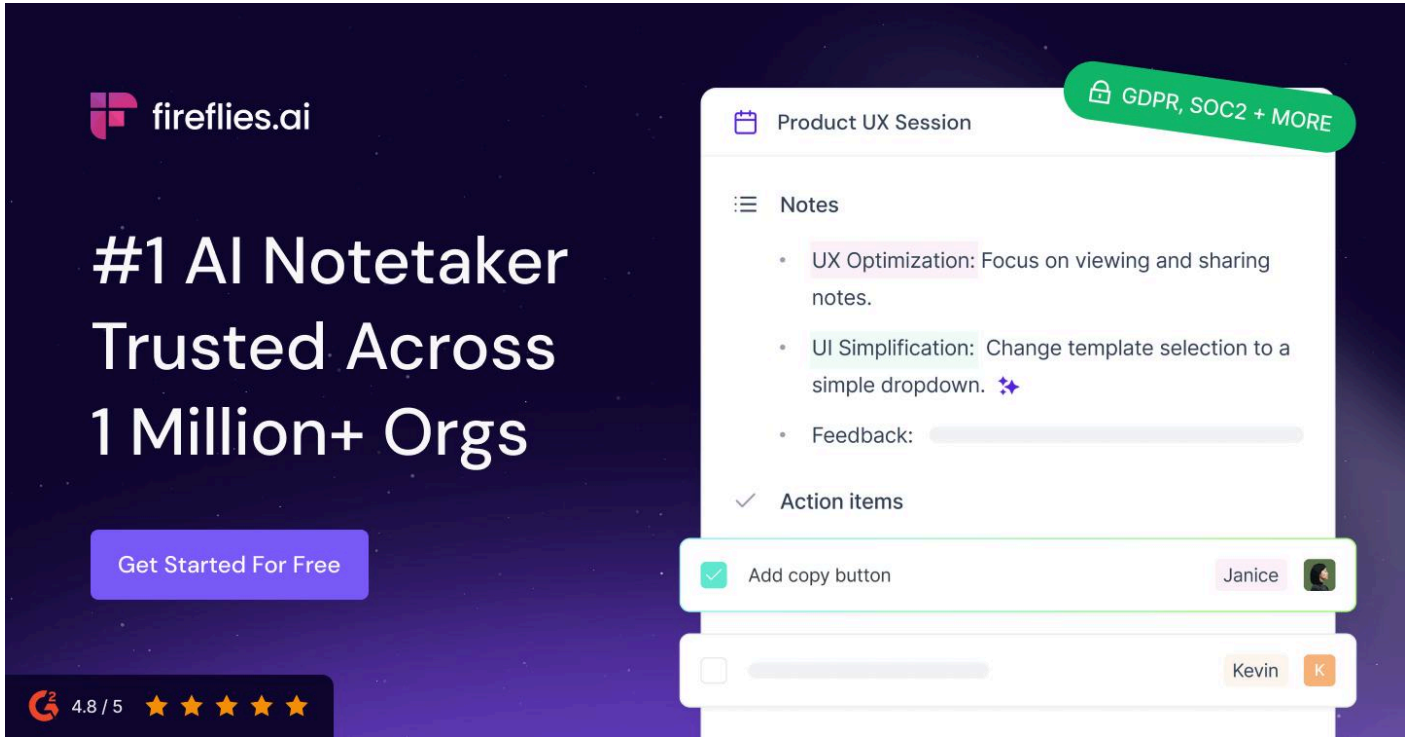
Landing page + ads — bán trước khi có sản phẩm



- Dựng landing page, chạy ads dù chưa có sản phẩm
- Khách hàng đăng ký mua → gọi điện xin lỗi rồi refund

Nguồn: Tony Xu (CEO DoorDash) kể lại trên Founders Podcast — davidsenra.com/episode/tony-xu.

AI Wizard of Oz MVP



This $1 billion AI startup founded by an MIT alum claimed to use AI, but its "AI" was just two founders taking notes by hand

Giả định rủi ro nhất:
"Người ta có trả tiền cho 'AI ghi chú họp' không?

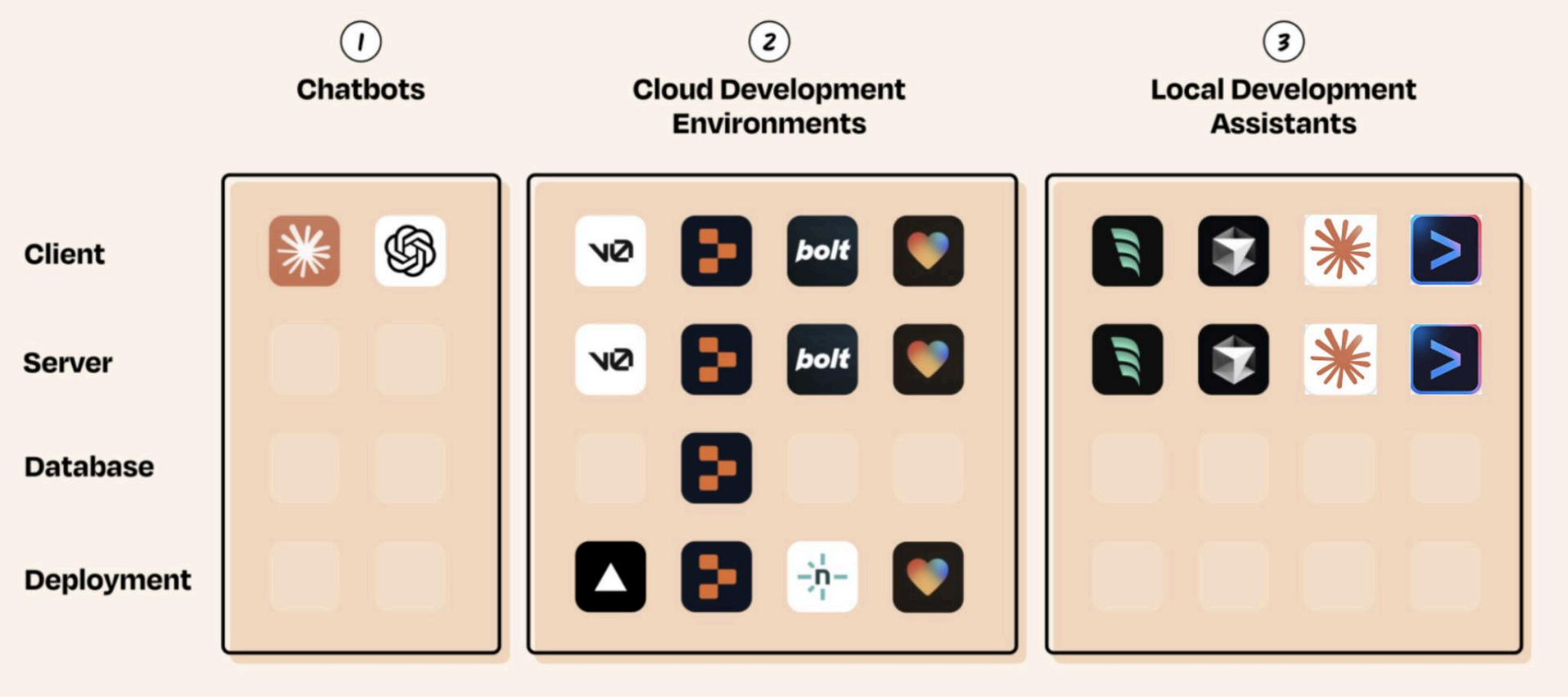
MVP:

- 2 founder tham gia vào cuộc họp dưới danh nghĩa một bot AI tên "Fred" (giả kiểu Siri), ngồi gõ note bằng tay.
- Làm tay hơn 100 cuộc họp, thu $100/tháng cho AI

Kết quả: seed $5M (10/2019) → kỳ lân >$1 tỷ (6/2025)

Nguồn: TechStartups, 12/11/2025 (headline trích nguyên văn) · Fireflies Blog — seed $5M (10/2019).

AI development tools



Phù hợp với: Prototype 1-vài trang, không có yêu cầu thiết kế quá phức tạp.

Phù hợp với: Prototype có nhiều hơn 1 tính năng, có yêu cầu thiết kế cụ thể, hoặc có nhiều trang / màn hình.

Phù hợp với: Người đã biết code và đang xây ứng dụng nghiêm túc, có mục tiêu đưa lên production.

Microsoft CPO: If you aren't prototyping with AI, you're doing it wrong

SECTION 02

Scope & PRD cho AI feature

Khung 6 bước của Ailian Gan (Lead PM AI, Zoom) — ví dụ Zoom meeting summary xuyên suốt

① Identify good use cases: AI là cây búa đi tìm cái đinh

Team thường được giao sẵn "cây búa" AI rồi mới đi tìm bài toán. Hãy tìm đinh tốt — đừng đóng lỗ lung tung.

A · HIỂU SÂU USER NEED

User làm những **hoạt động chính** nào trong product? **Pain point lớn nhất** là gì — chỗ nào complex hay tedious trong flow hiện tại? Điều gì sẽ **delight** user tới mức họ chưa từng nghĩ tới?

B · BRAINSTORM THEO LLM SKILLS

User need nào cải thiện được bằng các skill của LLM: **summarization · question-answering · content generation · personalization · data processing · predictive insights**. Nghĩ rộng: tóm tắt không chỉ PDF — còn chat, transcript, video.

C · CHỌN THEO GIÁ TRỊ + MOAT

Use case nào giải pain lớn nhất, tạo delight nhiều nhất? Cái nào tạo **competitive moat** — data nào (để train, để generate output) mà đối thủ có model tốt cũng **không copy được**?

Ví dụ: PM tại Zoom cân các use case

Scheduling — khả thi, nhưng cần biết availability & preferences của mọi người.

Draft agendas — khả thi, nhưng cần input data đủ tốt mới đề xuất được agenda hay.

Brainstorm ideas — khả thi, nhưng Zoom đã có sẵn sản phẩm whiteboard.

Extract takeaways — THẮNG. Zoom đã có sẵn transcript · LLM rất giỏi tóm tắt · app bên thứ ba đang bán note-taking → market demand rõ ràng.

PAIR Guidebook, ch. "User Needs + Defining Success" gọi đây là bước identify AI opportunities: bắt đầu từ user need, không bắt đầu từ công nghệ — câu hỏi đầu tiên luôn là "vấn đề này có thật sự cần AI không?"

Nguồn: Ailian Gan — "Write a PRD for a generative AI feature" (Reforge) · PAIR Guidebook, ch. "User Needs + Defining Success".

② Articulate the problem: đừng nhắc chữ AI

Problem statement KHÔNG được nhắc chữ AI — vấn đề của user không phải là "đời tôi thiếu AI". Hỏi: vấn đề là gì, vì sao là vấn đề, bạn biết từ đâu?

✕ BAD

"Users don't have an automated AI notetaker for all their meetings."

Phát biểu **thiếu solution = vấn đề**. Giả định user cần một AI notetaker — nhưng tại sao? Không mô tả vấn đề nền hay mục tiêu của user. Hỏi "why" vài lần để đào sâu hơn.

◐ GOOD

"Users want a record of the key points from their meetings, but it is tedious and distracting to take thorough manual notes for every meeting."

Tưởng tượng được **nhiều giải pháp**: thuê intern ghi note cho mọi cuộc họp? Bắt cả team ghi note chung? — AI chỉ là phương án scalable và rẻ hơn.

✓ EVEN BETTER

*"...a record of **discussion topics, key decisions, and action items**... In addition, users sometimes cannot attend a meeting, and they want a **quick way to catch up**."*

Gợi ý **nội dung cần focus** (decisions, action items), **use case phụ** (người vắng mặt cần xin note), và **cách consume** — phải nhanh, dễ đọc.

Nguồn: Ailian Gan — "Write a PRD for a generative AI feature" (Reforge) · ví dụ Zoom meeting summary.

③ Define goals: goals · non-goals · success metrics

Goals là outcome định tính gắn với problem statement — không phải solution, và cũng không nhắc AI. Ví dụ: Zoom meeting summary.

GOALS

· Đọc được recap **đầy đủ & chính xác** của cuộc họp — không cần đọc full transcript hay xem lại recording
· Nhận được list **action items + owners**
· Vắng mặt vẫn nắm được thông tin chính
· Truy cập summary theo cách hợp workflow
· **Edit summary** để sửa lỗi của LLM hoặc thêm context
· Quản lý được ai xem summary

Bài toán có thể giải tốt nhất mà không cần AI — goal không gắn với công nghệ.

NON-GOALS

· **Không** có customizable templates cho từng loại meeting

· **Không** bao gồm nội dung từ screen shares, chats, calendar invite hay tài liệu liên quan

Ghi rõ out-of-scope: ý tưởng cho version sau, vấn đề lâu đời feature này không giải, vấn đề kề cận cần xử lý riêng.

SUCCESS METRICS — 3 TẦNG

Usage
· % accounts enable base setting (setting **default off**)
· MAU của meeting summaries
· % meetings chạy summary

Quality
· % thumbs up vs thumbs down

Impact (khó đo → dùng proxy)
· Thời gian tiết kiệm khi không cần người ghi note
· % meetings user vắng mặt nhưng đọc summary

PAIR Guidebook, ch. "User Needs + Defining Success": define success từ sớm, trước khi scope giải pháp — success của AI feature không chỉ là accuracy mà là outcome cho user.

Nguồn: Ailian Gan — Reforge · ví dụ Zoom meeting summary · PAIR Guidebook, ch. "User Needs + Defining Success".

④a Scope the solution: mô tả user flow end-to-end

Bao gồm cả các bước KHÔNG có AI — đừng chỉ định nghĩa đoạn AI. Ví dụ: detect scheduling intent trong email xong → tự suggest giờ luôn, hay đưa user sang calendar app để hoàn tất?

Click a button

User bấm nút để invoke — ví dụ: summarize một chat thread.

One-shot prompt

Gõ prompt một lần vào text field — Notion brainstorm ý tưởng, Canva tạo graphic cho post.

Pre-set prompts

Prompt có sẵn để bấm — LinkedIn gợi ý takeaway questions trên mỗi post.

Automated report

Tự chạy theo lịch/sự kiện — Zoom meeting summary auto-start, Slack daily recap các chat bỏ lỡ.

Automated suggestions

Gợi ý hiện sẵn trong flow — Superhuman tóm tắt email 1 dòng, Vanta gợi ý trả lời questionnaire.

Chatbot

Hội thoại tự do — Intercom "Fin" trả lời support, Duolingo Roleplay luyện nói.

⚠ Đừng bắt đầu bằng chatbot

User gõ **gì cũng được** → quality khó kiểm soát; lại dính **cold start** — user không biết hỏi gì, hỏi thế nào, probe tiếp ra sao. Cân nhắc các interaction non-chatbot trước. **Figjam**: pre-set prompt gợi ý ở mỗi bước + vẫn cho gõ free text → khỏi viết prompt từ đầu, template chất lượng hơn.

Human-in-the-loop

Output có thể hallucinate → quyết định sai hoặc gây hại? Thêm bước **review / edit / delete trước khi share**. LLM soạn **draft** email — user sửa rồi mới gửi, không gửi ngay.

Nguồn: Ailian Gan — "Write a PRD for a generative AI feature" (Reforge).

④b AI-specific requirements — phần PRD không có ở feature thường

① USER INPUT + CONTEXTUAL DATA

Ai invoke LLM — bấm nút, gõ prompt, hay auto khi có sự kiện? LLM được đọc **dữ liệu nào** làm context — phải **định ranh giới data** rõ ràng:

· **Notion**: QA đọc cả document
· **Loom**: tạo title & chapters từ transcript
· **Intercom**: chatbot đọc knowledge base articles
· **M365 Copilot**: calendar + emails + docs + contacts

② LLM OUTPUT SPEC

Output non-deterministic → mô tả **length · tone · format · exclusion** độc lập ví dụ. "What good looks like": case **common** · **critical** phải đúng · **obscure** vẽ ranh giới. **Tip:** prototype câu trả lời bằng ChatGPT/Claude (upload transcript → generate).

Ví dụ Zoom — mục "Next Steps": chỉ action **sau** cuộc họp (không gồm việc trong meeting) · 1 action/bullet · có tên assignee · tối đa 8 · tone professional.

③ FEEDBACK MECHANISM

QA tốt đến đâu cũng không lường hết cách user dùng thật → cần kênh feedback:
Thumbs up/down — nhẹ, response rate cao, nhưng ít chi tiết vì sao tốt/xấu.
Form scoring + open text — nặng, response rate thấp, nhưng giàu chi tiết; ghi rõ data nào (input + output) được gửi kèm feedback.

PAIR Guidebook, ch. "Feedback + Control": feedback phải được thiết kế ngay trong spec, không gắn thêm sau khi ship — mỗi tương tác là cơ hội để hệ thống học.

④ QUALITY EVALUATION — BAR PHỤ THUỘC RISK

Có **HITL** check trước khi publish + output tiết kiệm nhiều thời gian → chấp nhận bar thấp hơn. Sai gây **quyết định tồi / hại**, long-tail offensive → bar cao hơn.
Auto eval: dataset mẫu → chạy task → auto score. **Manual eval**: PM/QA đọc mẫu bắt nuance. Tìm **CONVERGENCE** giữa hai cái.

Nguồn: Ailian Gan — "Write a PRD for a generative AI feature" (Reforge) · ví dụ Zoom meeting summary · PAIR Guidebook, ch. "Feedback + Control".

Nondeterminism là mặc định — không phải edge case

Ba loại failure của hệ xác suất — và ba sai lầm thiết kế khiến chúng gây thiệt hại.

① Output variance

Cùng một input, hai lần chạy ra hai output khác nhau. Đây là **hành vi mặc định** của mọi hệ probabilistic — không phải trường hợp ngoại lệ.

② Behavioral drift

Release thì đúng, vài tuần sau lệch — model update, input của user đổi, prompt gặp case chưa test. Team biết qua **complaint của user**, không qua monitoring của mình.

③ Reasoning-level failure

Retrieval đúng, tool call đúng, model trả lời trôi chảy — nhưng tổ hợp các bước ra kết quả sai. **"Monitoring shows all green. But the product fails."**

Sai lầm ① — Giấu variance

Không nút regenerate, không confidence framing → user báo "bug" cho hành vi đúng kỹ thuật. Hãy lộ ra: *"Here is one way to think about this"* + nút thử lại.

Sai lầm ② — Acceptance criteria nhị phân

"AI trả lời đúng" + 3 test case xanh → ship. Nhưng vài test case là **demo**, không phải **distribution** — nó giấu messy input và drift.

Sai lầm ③ — Fallback là ý sau cùng

Spec chỉ có 1 dòng "display error message". Nhưng failure trong hệ nondeterministic hiếm khi nhị phân — AI vẫn trả lời, chỉ là trả lời tệ, âm thầm bào mòn trust.

Nondeterminism không phải bug để sửa — là constraint để thiết kế vòng tránh, như latency hay kích thước màn hình.

Nguồn: Adaline Labs — "Designing AI Features for Nondeterminism" (Nilesh Barla, 28/3/2026).

Spec cho feature xác suất: 3 ca chuyển đổi

Mỗi ca chuyển đổi thay đổi thứ bạn ship — và thứ bạn đo được sau launch.

① Từ expected output → acceptance criteria dạng tỉ lệ

✕ "The AI returns a correct summary." → ✓ "The AI produces a summary that **passes this rubric on ≥90% of a representative input set.**" — Tỉ lệ đo được, và biết ngay khi nó xuống dốc.

② Từ test cases → test distributions

Một test case là demo. Một distribution mới là product. Bắt đầu bằng **~20 case phản ánh input thật** (messy, edge, ambiguous — không chỉ happy path), lớn dần từ **production traces**, không phải từ trực giác.

③ Từ "works" → "fails by design"

Spec phải có **Failure Modes section**: confidence thấp thì sao? tool timeout thì sao? output ngoài ngưỡng chấp nhận thì user thấy gì? Đây là quyết định product — viết vào spec, không phải thread Slack 3 tuần sau launch.

FALLBACK 3 TẦNG (chọn là quyết định product, ghi vào PRD):

Soft fallback — output đơn giản/hẹp hơn khi confidence thấp

Human handoff — case rủi ro cao/mơ hồ → chuyển người thật

Silent skip — không làm gì, nhưng cũng không làm sai

AI PRD thiếu acceptance threshold section = chưa phải AI PRD.

Nguồn: Adaline Labs — "Designing AI Features for Nondeterminism" (Nilesh Barla, 28/3/2026).

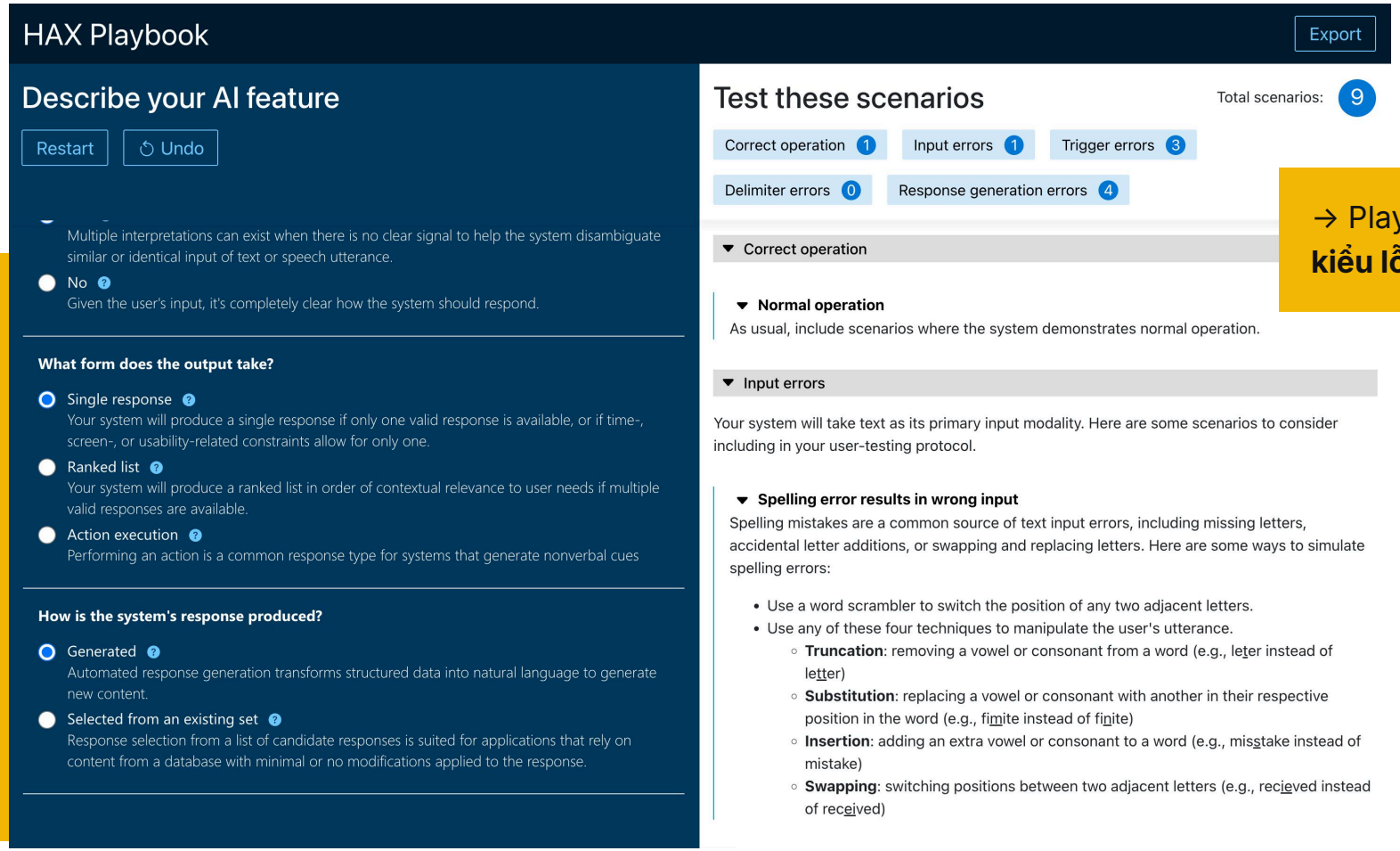
1. Mô tả tính năng AI

- Loại hệ thống
- Phương thức input:

Text/Voice

- Có trigger rõ ràng?
- Có phân định rõ input?
- Input có nhiều nghĩa
- Dạng output:
- Cách tạo response:

Generated/Selected



→ Playbook tự suy ra các **kiểu lỗi**, **kịch bản cần test**

Microsoft HAX Playbook Github

github.com/microsoft/HAXPlaybook

*Playbook giúp các nhóm phát triển sản phẩm AI xác định những **kịch bản kiểm thử quan trọng** cần thực hiện trước khi ra mắt tính năng AI, dựa trên **đặc điểm cụ thể** của hệ thống đó.*

Case BatchBuddy: bản đồ 9 kịch bản theo lớp lỗi

Chatbot kênh #batch02-general · source of truth: deadline **20:00 22/06/2026**

config · trigger: **bot tự đoán khi nào nói**

Correct operation — 1

đường chạy đúng

- User hỏi deadline → bot trả đúng **20:00 22/06**
- Không nói khi không cần

Input errors — 1

typo trong câu hỏi

- truncation
- substitution
- insertion
- swapping

Trigger errors — 3

bot tự đoán khi nào nói

- **missed** — đáng nói thì im
- **spurious** — không ai hỏi vẫn nói
- **delayed** — nói quá trễ

Delimiter errors — 0

"0" cũng là một câu trả lời có ý nghĩa — lớp này được xét và loại trừ, không phải bị quên.

Response generation — 4

sinh câu trả lời sai

- ambiguities
- implausible
- **plausible-but-incorrect**
- inappropriate

⚠ Ví dụ SCN-08 — plausible-but-incorrect

User hỏi deadline, bot trả **"18:00 22/06"** — nghe hợp lý nhưng **sai** (đúng: 20:00). Kiểu sai nguy hiểm nhất: hợp lý nên khó bị phát hiện.

Nếu có nút bấm để hỏi thay vì tự đoán → 3 lỗi trigger biến mất. Cấu hình quyết định bề mặt rủi ro.

④c Privacy · ⑤ Align engineering · ⑥ GTM

Ba phần cuối của PRD cho AI feature — vẫn lấy ví dụ Zoom AI Companion / meeting summary.

④c PRIVACY & CONTROLS

· **Disclosure:** banner pop-up cho mọi participant khi AI bắt đầu chạy + **sparkle indicator** hiển thị suốt lúc summary đang chạy
· Base setting **default off** tới khi account admin enable
· **Kill switch:** host dừng summary bất cứ lúc nào, xóa luôn transcript tương ứng
· Transcript retention tối đa **30 ngày**
· Data của khách **KHÔNG** dùng để train model

⑤ ALIGN ENGINEERING

Đừng over-spec. Thảo luận với engineering ngay khi draft spec — đừng chờ spec hoàn chỉnh; sẽ có nhiều unknown về việc LLM làm được/không làm được.

PM cần hiểu đủ để cân quyết định product:
· **Prompting** — prompt user gửi thẳng hay lồng trong prompt lớn hơn
· **Model selection** — 1st vs 3rd party, privacy
· **LLM techniques** — fine-tuning, RAG
· **Scaling** — GPU đắt, capacity, data residency

⑥ GTM

Rollout theo tier (beta trước để monitor quality): nội bộ → premium tier → GA

Pricing: add-on SKU hay bundled? Zoom: **free kèm meetings license** — muốn AI phổ biến rộng, không để IT admin phải chọn ai được dùng AI.

Enablement: training deck về behavior + rollout + privacy · FAQ · channel chat **"Ask AI Companion"** cho field team hỏi đáp.

Nguồn: Ailian Gan — "Write a PRD for a generative AI feature" (Reforge) · ví dụ Zoom AI Companion.

SECTION 03

Human-centered AI design

Thiết kế AI lấy con người làm trung tâm

Phần mềm truyền thống vs AI product

Phần mềm chạy theo luật. AI chạy theo xác suất — và xác suất nghĩa là sẽ có lúc sai.

	PHẦN MỀM TRUYỀN THỐNG	AI PRODUCT
Kết quả	Luôn giống nhau	Mỗi lần một khác
Ví dụ	"Số dư của tôi?" → trả đúng số, mọi lúc	"Tóm tắt email này" → mỗi lần một bản
Kiểm thử	Đạt / không đạt	Chạy 100 lần → bao nhiêu lần "đủ tốt"?
Lỗi	Bug — tìm và sửa được	Sai xác suất — không sửa được, chỉ giảm được

> *"Bạn không biết user sẽ tương tác thế nào, và cũng không biết LLM sẽ phản hồi ra sao — input, output, process, cả ba đều không chắc chắn."* — **Aishwarya (Ashi) Naresh Reganti, Lenny's Newsletter 1/2026**

Nếu sản phẩm dùng AI, bạn đang thiết kế cho uncertainty.

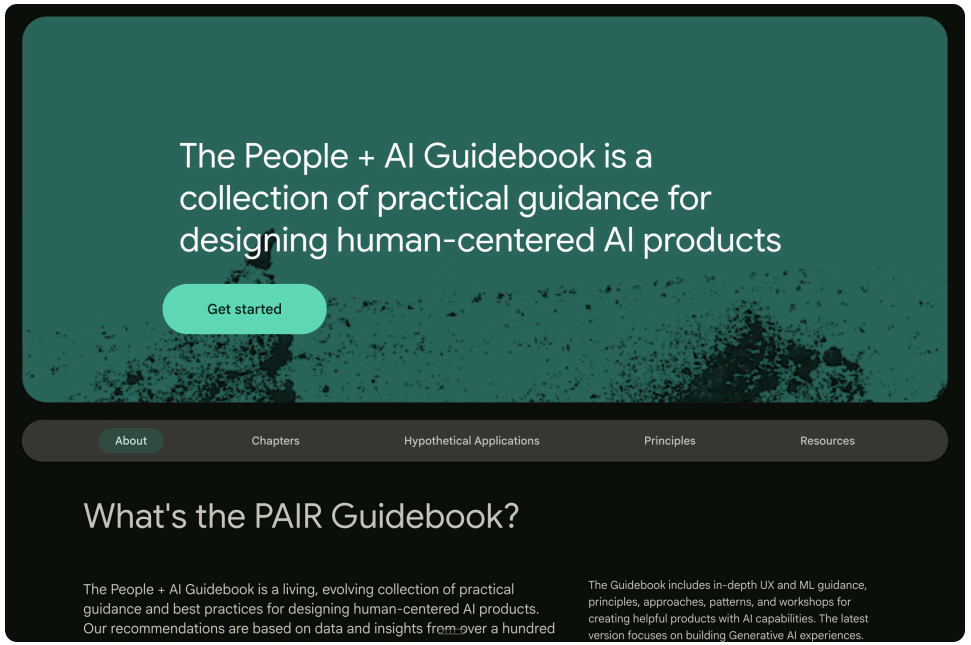
Nguồn: Aishwarya Naresh Reganti (ex-AWS · LevelUp Labs) & Kiriti Badam (OpenAI) — Lenny's Newsletter, 11/1/2026 (diễn giải ý).

Những câu hỏi thiết kế dành cho sản phẩm AI

- Mình đã đặt đúng kỳ vọng cho người dùng chưa?
- Làm thế nào chúng ta biết được AI có thể làm gì, không thể làm gì và cách nó sẽ mắc lỗi?
- Thiết kế phản hồi thế nào khi AI sai?
- Xây dựng vòng lặp feedback

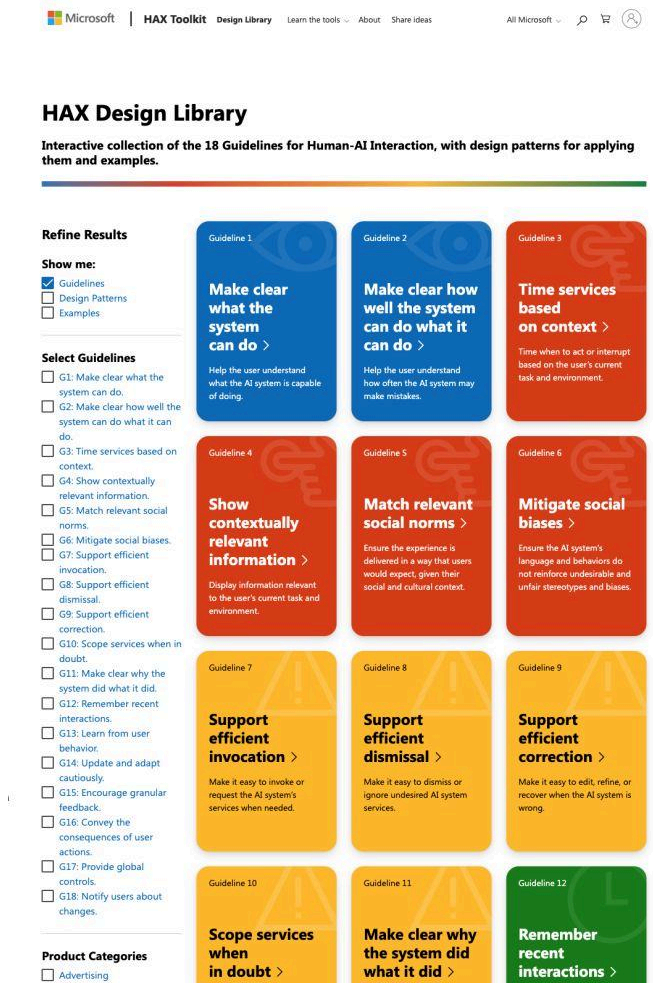
Tài liệu buổi học

PAIR Guidebook (Google)



*PAIR thiên về **AI product framing**: chọn đúng bài toán AI, định nghĩa success, trust, feedback, graceful failure.*

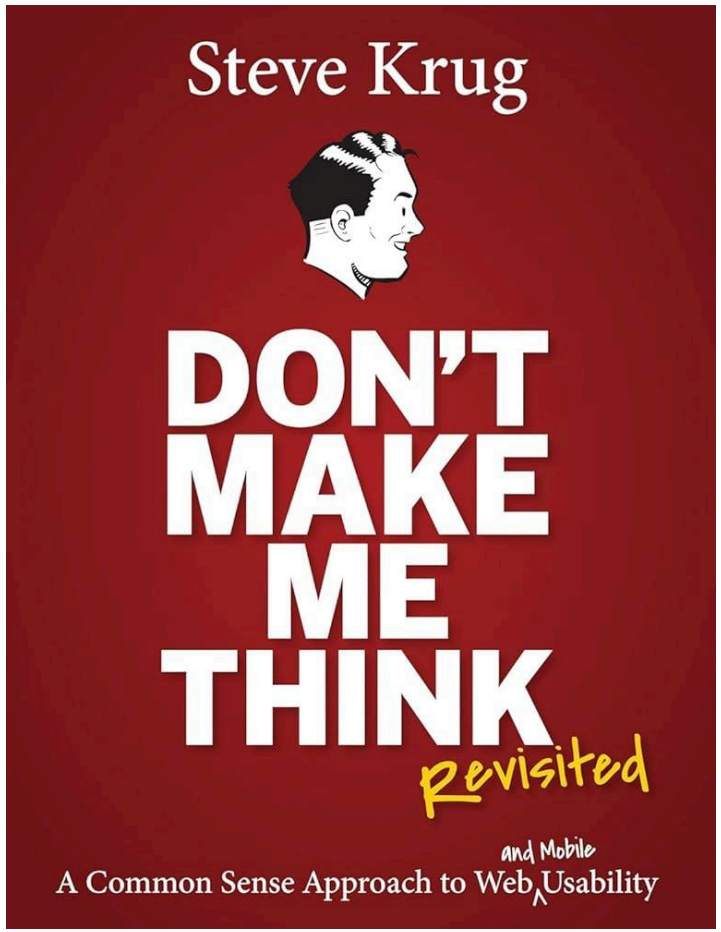
HAX Toolkit (Microsoft)



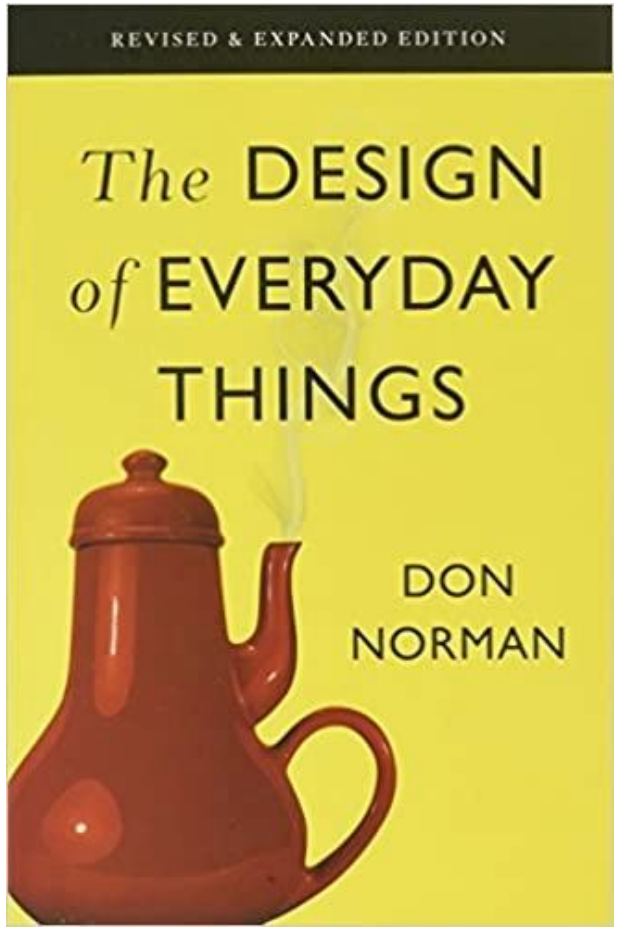
*HAX thiên về **AI interaction design**: guideline, pattern, planning, và test scenario cho những failure của AI*

PAIR: pair.withgoogle.com · HAX Playbook: github.com/microsoft/HAXPlaybook

Sách nên đọc



*Krug thiên về **how**: làm sao để interface bớt rối, bớt bắt người dùng dừng lại để suy nghĩ.*



*Norman thiên về **why**: vì sao user không hiểu hệ thống, vì sao feedback, mapping và conceptual model lại quan trọng*

Link sách

Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts

J.D. Zamfirescu-Pereira
zamfi@berkeley.edu
UC Berkeley
Berkeley, CA, USA

Richmond Wong
rwong34@gatech.edu
Georgia Institute of Technology
Atlanta, Georgia, USA

Bjoern Hartmann
bjoern@eecs.berkeley.edu
UC Berkeley
Berkeley, CA, USA

Qian Yang
qianyang@cornell.edu
Cornell University
Ithaca, NY, USA

ABSTRACT

Pre-trained large language models ("LLMs") like GPT-3 can engage in fluent, multi-turn instruction-taking out-of-the-box, making them attractive materials for designing natural language interactions. Using natural language to steer LLM outputs ("prompting") has emerged as an important design technique potentially accessible to non-AI-experts. Crafting effective prompts can be challenging, however, and prompt-based interactions are brittle. Here, we explore whether non-AI-experts can successfully engage in "end-user prompt engineering" using a design probe—a prototype LLM-based chatbot design tool supporting development and systematic evaluation of prompting strategies. Ultimately, our probe participants explored prompt designs opportunistically, not systematically, and struggled in ways echoing end-user programming systems and interactive machine learning systems. Expectations stemming from human-to-human instructional experiences, and a tendency to overgeneralize, were barriers to effective prompt design. These findings have implications for non-AI-expert-facing LLM-based tool design and for improving LLM-and-prompt literacy among programmers and the public, and present opportunities for further research.

1 INTRODUCTION

The idea of instructing computers in natural language has fascinated researchers for decades, as it promises to make the power of computing more customizable and accessible to people without programming training [4]. The combination of pre-trained large language models (LLMs) and prompts brought renewed excitement to this vision. Recent pre-trained LLMs (e.g., GPT-3 [8], ChatGPT [1]) can engage in fluent, multi-turn conversations out-of-the-box, substantially lowering the data and programming-skill barriers to creating passable conversational user experiences [7]. People can improve LLM outputs by prepending *prompts*—textual instructions and examples of their desired interactions—to LLM inputs. Prompts directly bias the model towards generating the desired outputs, raising the ceiling of what conversational UX is achievable for non-AI experts. In the past two years, social media platforms have witnessed an explosion of posts showing the results of lay peoples' experimentation with LLMs for question answering, creative dialogue writing, writing code, and more. This excitement around LLMs and prompting is propelling a rapidly growing set of LLM-powered applications [23] and prompt design tools [3, 20, 32].

Yet despite widespread excitement, surprisingly little is known

Why Johnny Can't Prompt [CHI 2023]

- Người dùng viết prompt thường **không biết AI làm được gì / không làm được gì**. Vì vậy họ cần ví dụ hoặc chỉ dẫn cụ thể để biết nên tiếp cận thế nào.
- Người dùng viết prompt thường **khái quát hóa quá mức** từ chỉ một vài ví dụ hoặc một vài lỗi nhỏ (và dễ bỏ cuộc sớm).
- Một số người **kỳ vọng AI sẽ hiểu chỉ dẫn giống như** con người hiểu

Zamfirescu-Pereira, J.D., et al. "Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts." CHI 2023 — people.eecs.berkeley.edu/~bjoern/papers/zamfirescu-johnny-chi2023.pdf

Đừng hứa hẹn quá khả năng của AI

*Đây là vấn đề về **kỳ vọng**, không chỉ là vấn đề về độ chính xác*

“Don't let your UI write a check that your AI can't cash.”

- Eytan Adar (2018)

✕ Auto-resolve customer issue vs ✓ Draft reply for human review

1
INITIALLY

Make clear what the system can do ⓘ

Help the user understand what the AI system is capable of doing.

Guidelines for Human-AI Interaction (Microsoft)

PAIR Guidebook, ch. "Mental Models" — cùng một ý: UI là nơi đặt kỳ vọng.

Guidelines for Human-AI Interaction (Microsoft)

Bộ 18 nguyên tắc thiết kế AI UX theo 4 chặng của trải nghiệm người dùng

Amershi et al., "Guidelines for Human-AI Interaction", CHI 2019 — microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction · HAX Playbook: github.com/microsoft/HAXPlaybook

CẤU HÌNH	CASE 1 · CHỌN TEMPLATE	CASE 2 · SINH TỰ ĐỘNG
Lĩnh vực hệ thống	Dự đoán và hỗ trợ văn bản	Dự đoán và hỗ trợ văn bản
Phương thức đầu vào	Văn bản	Văn bản
Kích hoạt tự động	Có	Có
Dấu phân cách	Có	Có
Diễn giải ngữ cảnh	Không	Không
Kiểu phản hồi	Danh sách xếp hạng	Danh sách xếp hạng
★ **Cơ chế phản hồi**	**Chọn template**	**Sinh tự động**

★ Chỉ dòng **Cơ chế phản hồi** thay đổi. Toàn bộ 6 config còn lại giữ nguyên.

Email Assistant

AI gợi ý reply khi nhận email mới

→ Đổi cách AI tạo câu trả lời = đổi luôn nghĩa vụ thiết kế và kiểm soát của team.

Case 1 · Chọn template 6 kịch bản

HOẠT ĐỘNG ĐÚNG

- ✅ Correct operation
  Hoạt động bình thường

LỖI ĐẦU VÀO

- ⚠️ Spelling error results in wrong input
  Lỗi chính tả dẫn đến đầu vào sai

LỖI SINH PHẢN HỒI

- 🔴 Poor precision
  Độ chính xác thấp
- 🔴 Poor recall
  Độ bao phủ thấp
- 🔴 Poor ranking
  Xếp hạng sai
- 🔴 Low result diversity
  Kết quả thiếu đa dạng

không còn ở case 2 nữa

Kiểu lỗi: truy xuất / xếp hạng — hệ thống chọn nhầm hoặc xếp thứ tự sai từ tập template có sẵn.

Case 2 · Sinh tự động 7 kịch bản

HOẠT ĐỘNG ĐÚNG

- ✅ Correct operation
  Hoạt động bình thường

LỖI ĐẦU VÀO

- ⚠️ Spelling error results in wrong input
  Lỗi chính tả dẫn đến đầu vào sai

LỖI SINH PHẢN HỒI

- 🔴 **Implausible or incoherent** MỚI
  Output vô lý hoặc mất logic nội tại
- 🔴 **Plausible but incorrect** MỚI
  Output nghe có lý nhưng sai thực tế (ảo giác)
- 🔴 **Inappropriate** MỚI
  Output sai về tone, ngữ cảnh, hoặc vi phạm chuẩn mực
- 🔴 Poor ranking
  Xếp hạng sai
- 🔴 Low result diversity
  Kết quả thiếu đa dạng

Kiểu lỗi: chất lượng sinh nội dung — AI có thể bịa, sai thực tế, hoặc sai giọng. Không còn là "chọn nhầm" nữa.

Trust calibration

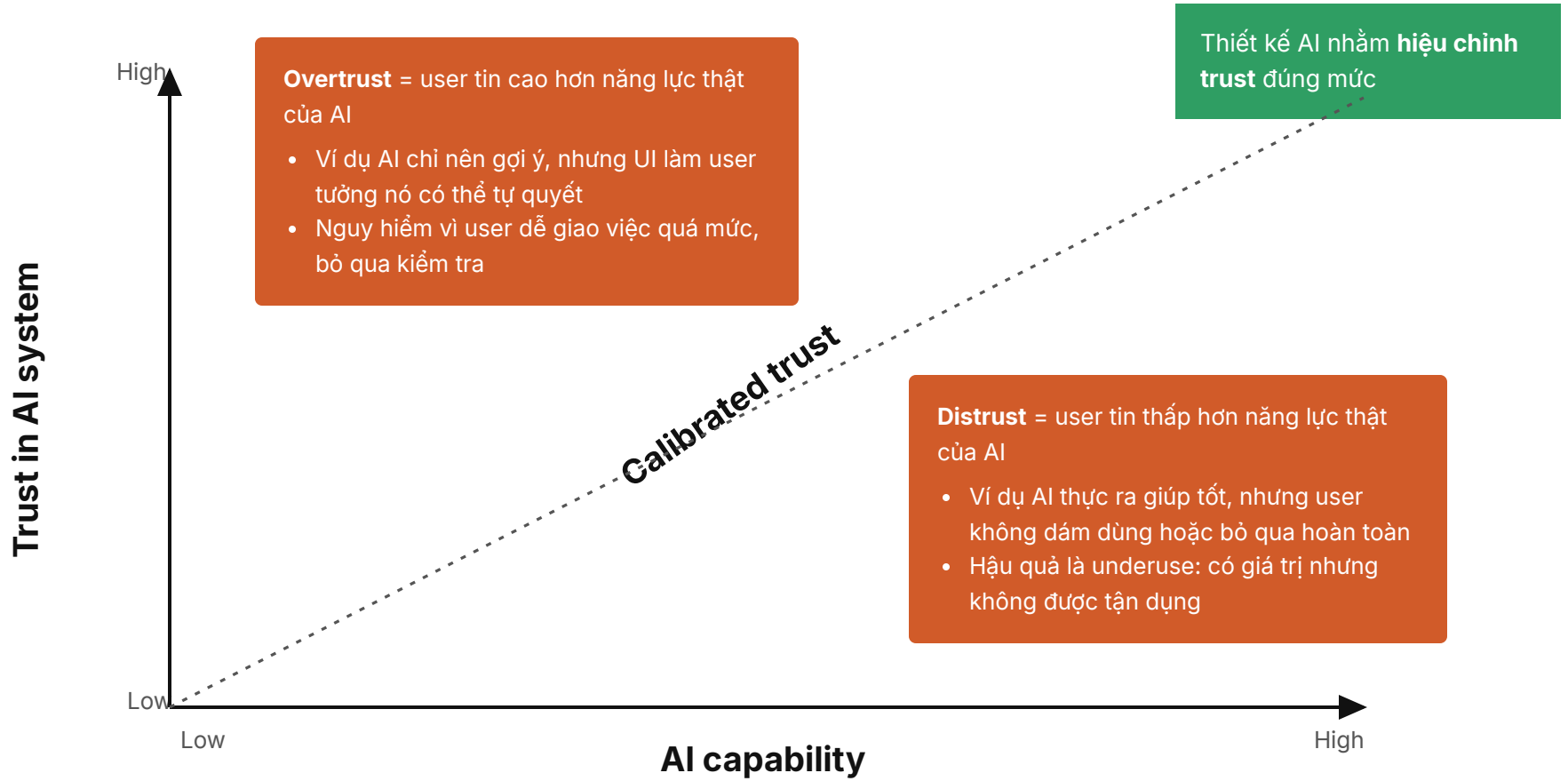


Figure 4-1. Trust calibration. *Users can overtrust the AI when their trust exceeds the system's capabilities. They can distrust the system if they are not confident of the AI's performance*

Source: Designing Human-Centric AI Experiences Applied UX Design for Artificial Intelligence (Akshay Kore)

Trust calibration = expectation + explainability + control

Expectation

Nói rõ AI làm được gì, làm tốt tới đâu, khi nào dễ sai.

Explainability

Giúp user hiểu vì sao AI ra output này và khi nào nên nghi ngờ.

Control

Cho user sửa, bỏ qua, undo, preview, hoặc duyệt trước khi commit.

Ba phần còn lại của deck thực ra chính là unpack ba trụ này.

Augmentation vs Automation

Chọn mức độ tự chủ (agency) theo độ chắc chắn và chi phí khi sai.

Augmentation

- Cần phán đoán, sáng tạo hoặc sở thích cá nhân.
- Ý định và yêu cầu của user còn mơ hồ
- User phải chịu trách nhiệm cho quyết định cuối cùng.
- Sai sót có hậu quả lớn, khó phục hồi
- Workflow kéo dài, nhiều bước, thường xuyên thay đổi.

Automation

- Lặp lại, tốn thời gian hoặc ít giá trị sáng tạo.
- Quy trình rõ ràng; input và output dễ xác định.
- Hệ thống có thể thực hiện ổn định với ít giám sát.
- Nếu AI sai, hậu quả thấp, dễ phát hiện hoặc dễ hoàn tác.
- User muốn giao việc vì không đủ thời gian hoặc nguồn lực.

Copilot: ~30% acceptance rate mà 4.7M paid users (1/2026) — augmentation đúng chuẩn (GitHub · Microsoft earnings).

Augmentation vs Automation

Chọn mức độ tự chủ (agency) theo độ chắc chắn và chi phí khi sai.

Augmentation → **Automation**

Inaction

- Chưa đủ chắc user muốn gì
- Sai một lần là rất đắt
- Giữ quyền quyết định cho người dùng

Ask

- Có tín hiệu đúng, nhưng còn mơ hồ
- Một câu hỏi ngắn giảm rủi ro lớn
- Hợp với việc quan trọng nhưng chưa rõ ý

Act

- Đủ chắc user muốn gì
- Làm sai vẫn dễ sửa hoặc undo
- Tự động hóa để tiết kiệm thời gian

Mixed initiative như một bài toán quyết định giữa act, ask, hay not act dựa trên expected value, chứ không chỉ là sở thích thiết kế. **Ask** thường là trạng thái thông minh nhất nhưng lại hay bị bỏ quên nhất.

Eric Horvitz, Principles of Mixed-Initiative User Interfaces (CHI 1999)

Thiết kế độ tự chủ (agency) theo chi phí khi sai (cost-of-error)

KHÔNG LÀM HỎI TỰ LÀM

Trục → 0 —— P(goal) —— 1

① Sai mà ĐẮT xóa dữ liệu · gửi nhầm tiền

KHÔNG LÀM **(rộng)** HỎI **(rộng)**

→ vùng **TỰ LÀM co lại** ở mép phải — phải **RẤT chắc** mới dám act

---

② Sai mà RẺ gợi ý autocomplete · undo được

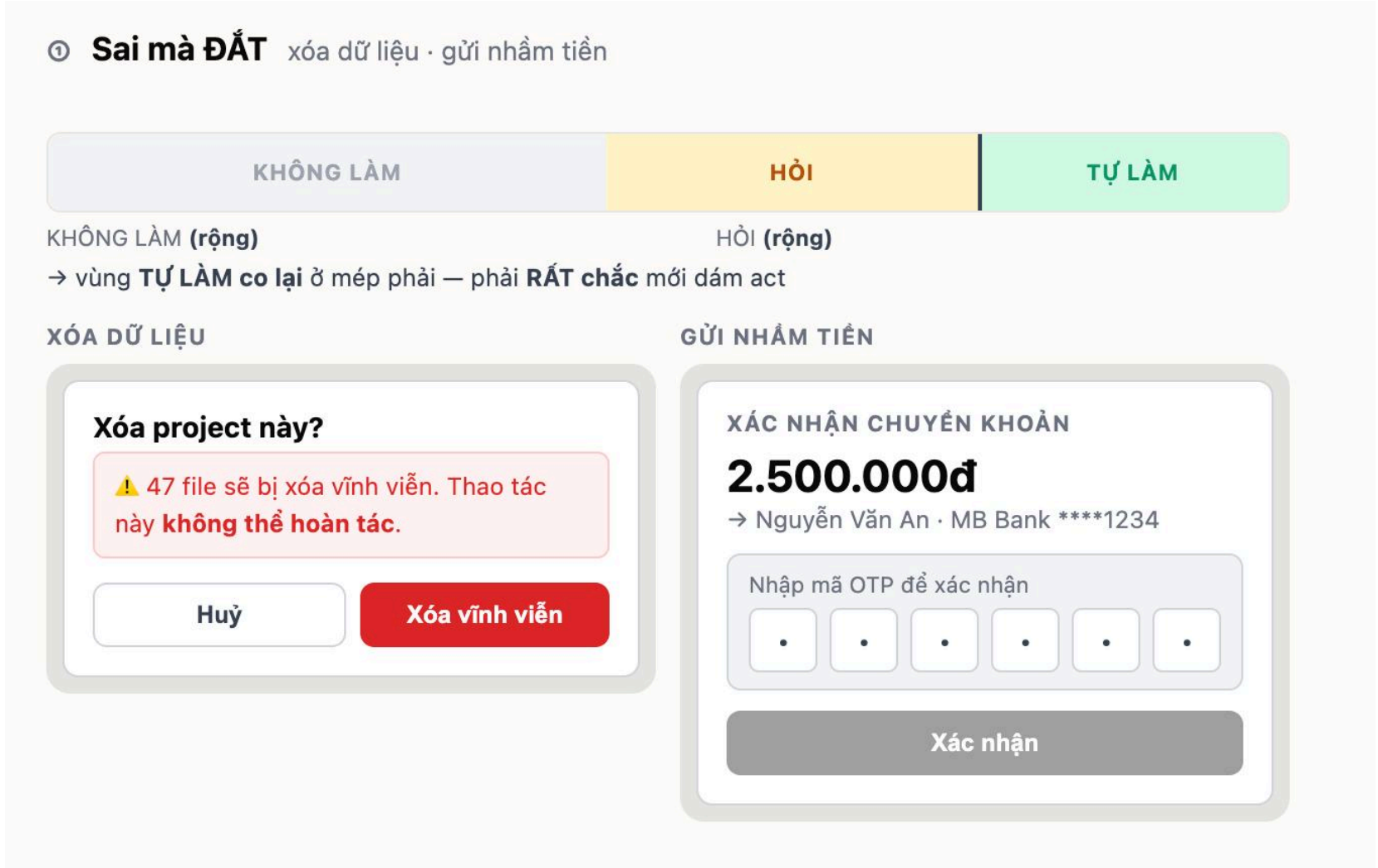
KHÔNG HỎI TỰ LÀM **(rộng gần hết trục)**

→ vùng **TỰ LÀM nở ra** — cứ act, hỏng cũng undo được

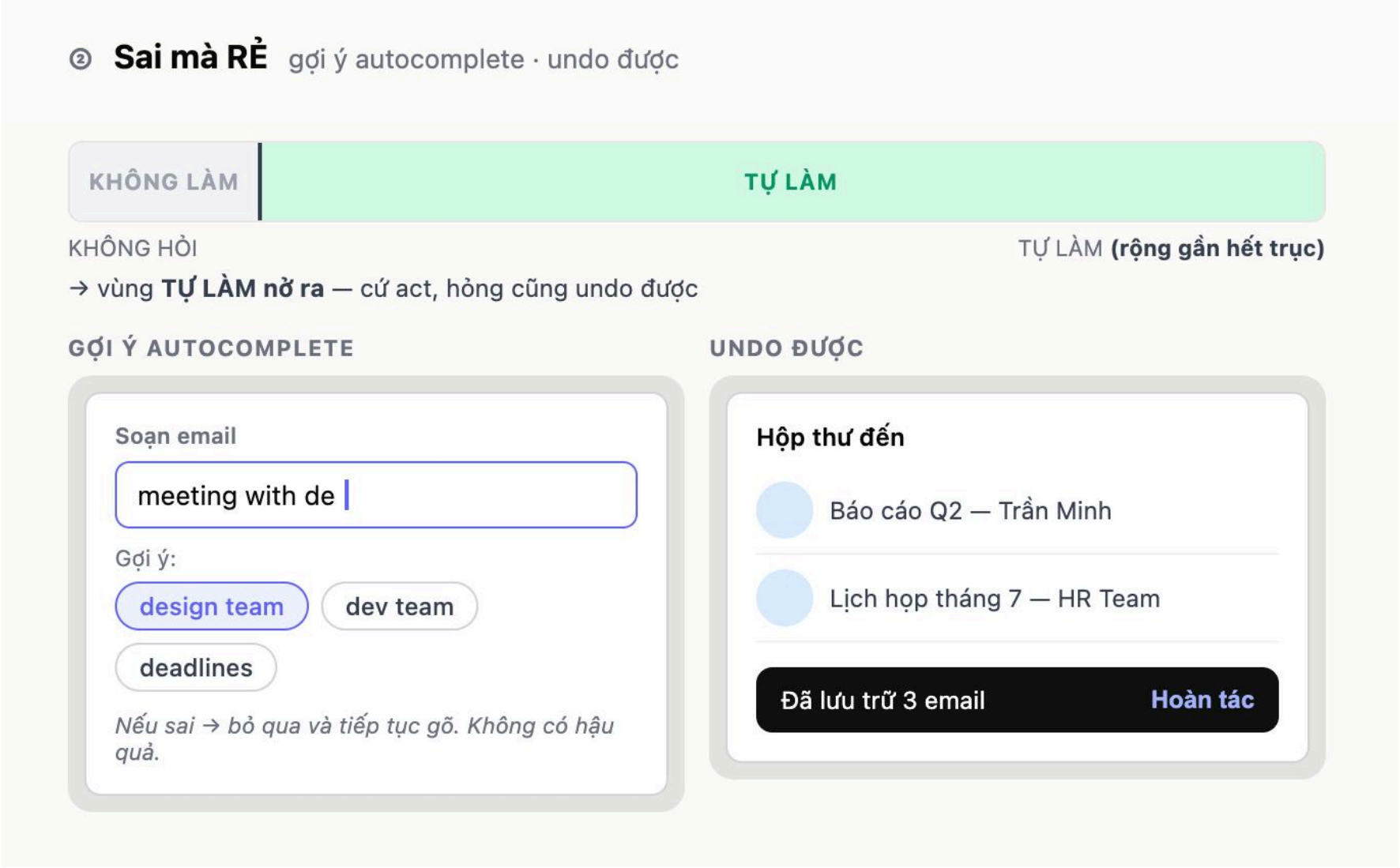
Chốt: cùng một độ "chắc" (cùng điểm trên trục) — nhưng ngưỡng để **TỰ LÀM dịch theo CÁI GIÁ KHI SAI**.

⟹ *agency = độ-chắc × cost-of-error*, **không phải slider cố định**.

Thiết kế độ tự chủ (agency) theo chi phí khi sai (cost-of-error)



Thiết kế độ tự chủ (agency) theo chi phí khi sai (cost-of-error)



Làm rõ AI làm được gì

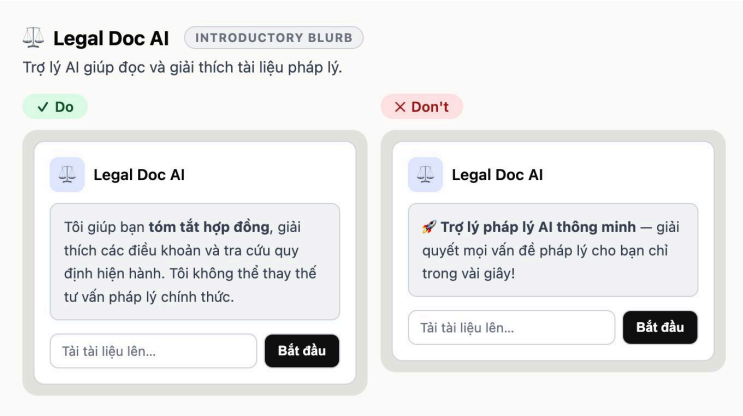
Source: Make clear what the system can do

Nên dùng khi:

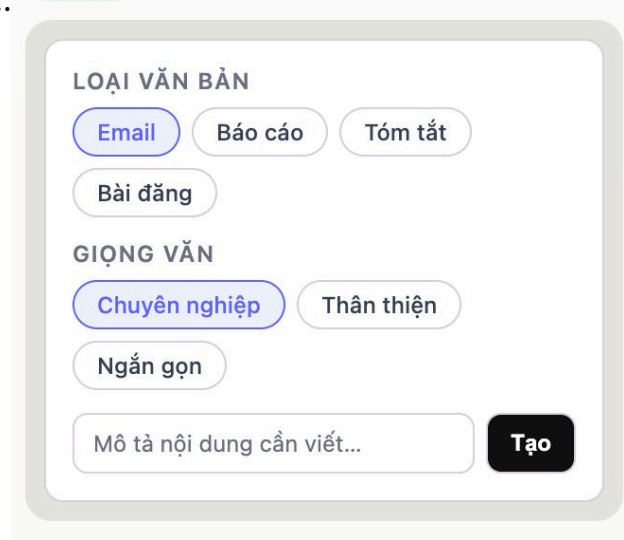
- Người dùng chưa quen với loại AI này.
- Tính năng mới hoặc khó tự khám phá.
- Hệ thống có nhiều khả năng nhưng không dễ nhìn ra ngay.
- Muốn hướng người dùng vào những kiểu đầu vào mà AI xử lý tốt hơn.

Design patterns:

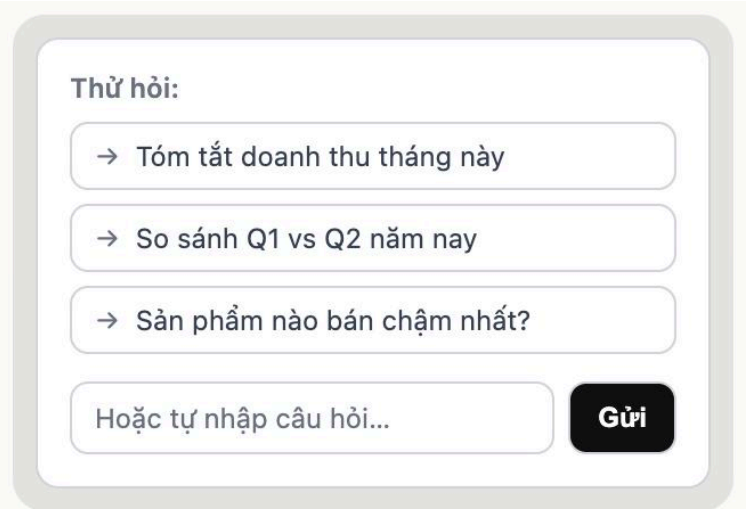
Use explanations: Dùng phần giải thích để người dùng hiểu hệ thống có thể làm gì



Expose system controls: Làm lộ các nút, menu, tùy chọn, hoặc cài đặt để người dùng nhìn vào là hiểu hệ thống có những khả năng nào...



Demonstrate possible inputs: Cho ví dụ prompt, ví dụ câu hỏi, hoặc gợi ý đầu vào để người dùng thấy được những dạng tương tác...



Capability cue là một phần của interaction design — không chỉ là onboarding copy.

Làm rõ hệ thống làm tốt đến đâu

Source: Make clear how well the system can do what it can do

Hệ thống cần giúp user hình dung đúng về mức độ chính xác, độ ổn định, và những tình huống AI có thể sai.

🌿 Flora

✓ Do

Nói rõ: làm gì + giới hạn scope. User biết ngay mình đang dùng tool gì.

✕ Don't

Chỉ tên + slogan. Không biết: chụp để làm gì, nhận diện loại nào, giới hạn ra sao.

🍝 ChefAI

✓ Do

Rõ: làm được gì (công thức), chưa làm được gì (đồ uống, bánh). Không over-promise.

✕ Don't

Tự nhận 'số 1'. Không nói gì về chức năng thực. User không biết nên chụp gì.

✈️ Trip Planner AI

Gợi ý lịch trình & giá khách sạn theo ngân sách.

✓ Do Giá tham khảo tại thời điểm gợi ý — "giá thực tế có thể thay đổi khi đặt".

✕ Don't "Đây là lựa chọn phù hợp nhất" + tổng chi phí 4.200.000đ nói như chắc chắn.

🍱 Food Scanner

Chụp món ăn, AI ước tính calo và ghi nhật ký.

✓ Do Khoảng "350–430 kcal" + nói rõ là ước tính vì ảnh hơi mờ.

✕ Don't Một con số exact "387 kcal" — precision giả.

📄 Resume Screener

AI xếp hạng CV, hỗ trợ HR ra quyết định sàng lọc.

✓ Do "Danh sách phù hợp để bạn xem trước" + nêu giới hạn dữ liệu đầu vào.

✕ Don't Auto loại ứng viên ("Đã loại") trước khi con người xem.

🧠 Mental Health Journaling AI

Đọc nhật ký, nhận diện cảm xúc, gợi ý reflection.

✓ Do "Đây là quan sát từ nhật ký, không phải chẩn đoán".

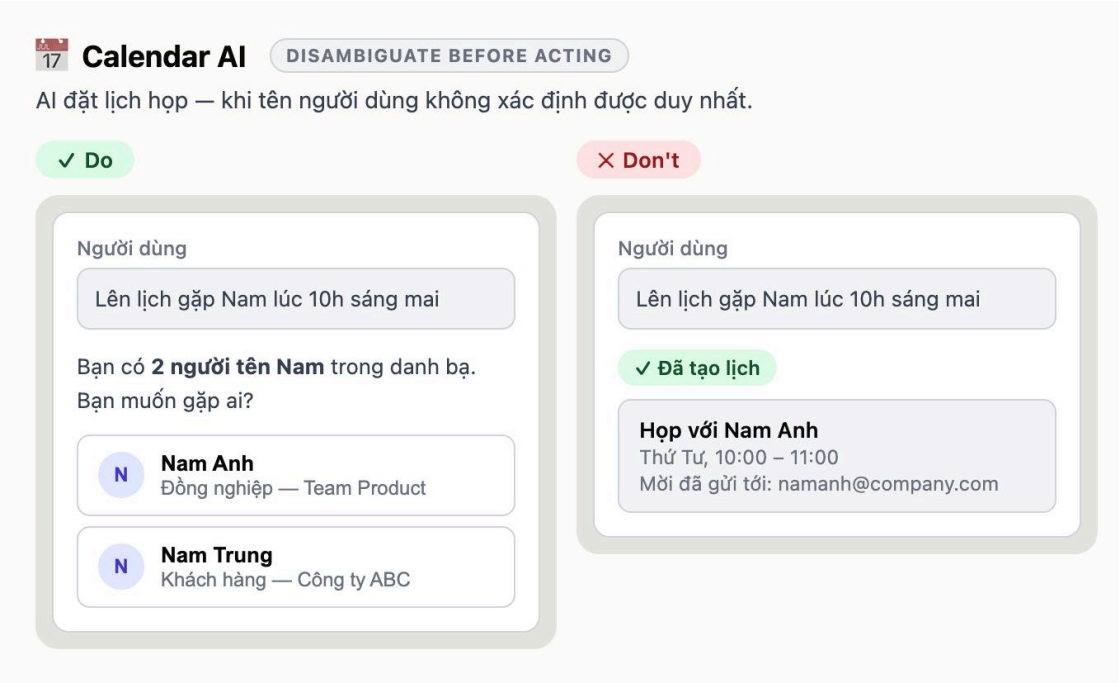
✕ Don't Gắn nhãn "Burnout giai đoạn 2" như chẩn đoán chắc chắn.

Mỗi domain cần một cách nói khác nhau — precision giả là một lỗi thiết kế.

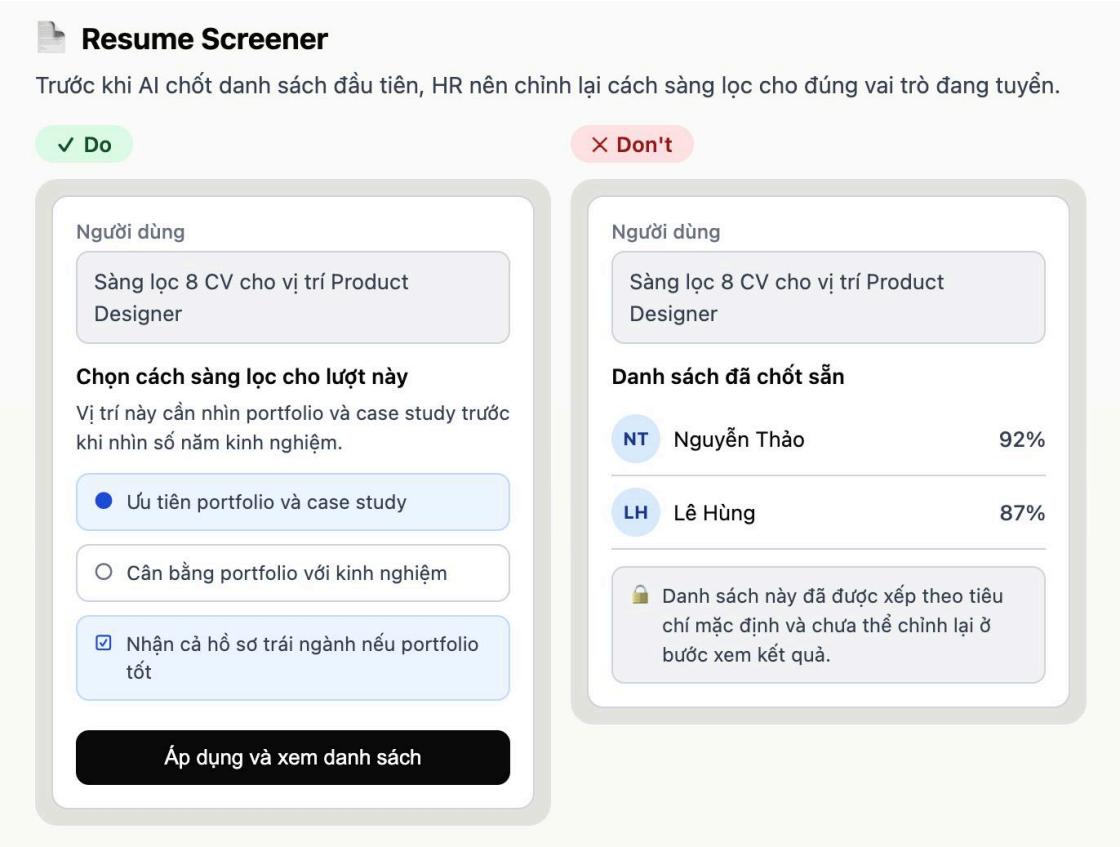
Khi AI sai / không chắc chắn

Source: Scope services when in doubt

Hỏi lại trước khi làm



Cho phép user tuỳ chỉnh



Khi AI không chắc, bớt làm đi thường là UX tốt hơn.

Khi AI không chắc để hiểu người dùng, nó nên hỏi lại — thay vì cố giải quyết một câu hỏi mơ hồ.

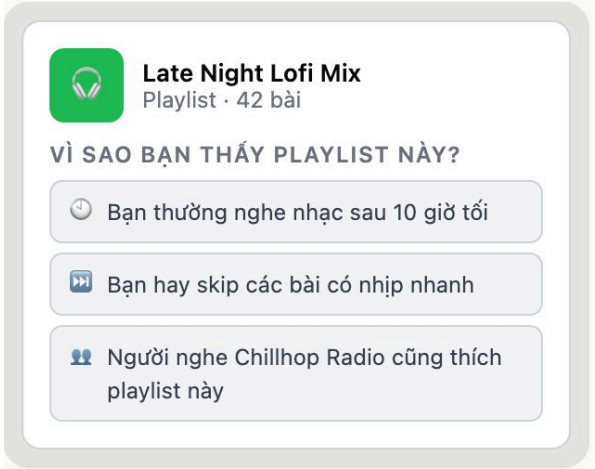
HAX G10 "Scope services when in doubt" (Amershi et al., CHI 2019) · D18 HCAI (batch02 day18).

Giải thích vì sao hệ thống làm như vậy

Người dùng cần hiểu tại sao khi kết quả ảnh hưởng quyết định, hoặc có vẻ "khó hiểu"

Giải thích lý do đưa ra quyết định

Map user behaviors to system outputs: hành vi trước đây ảnh hưởng đầu ra thế nào



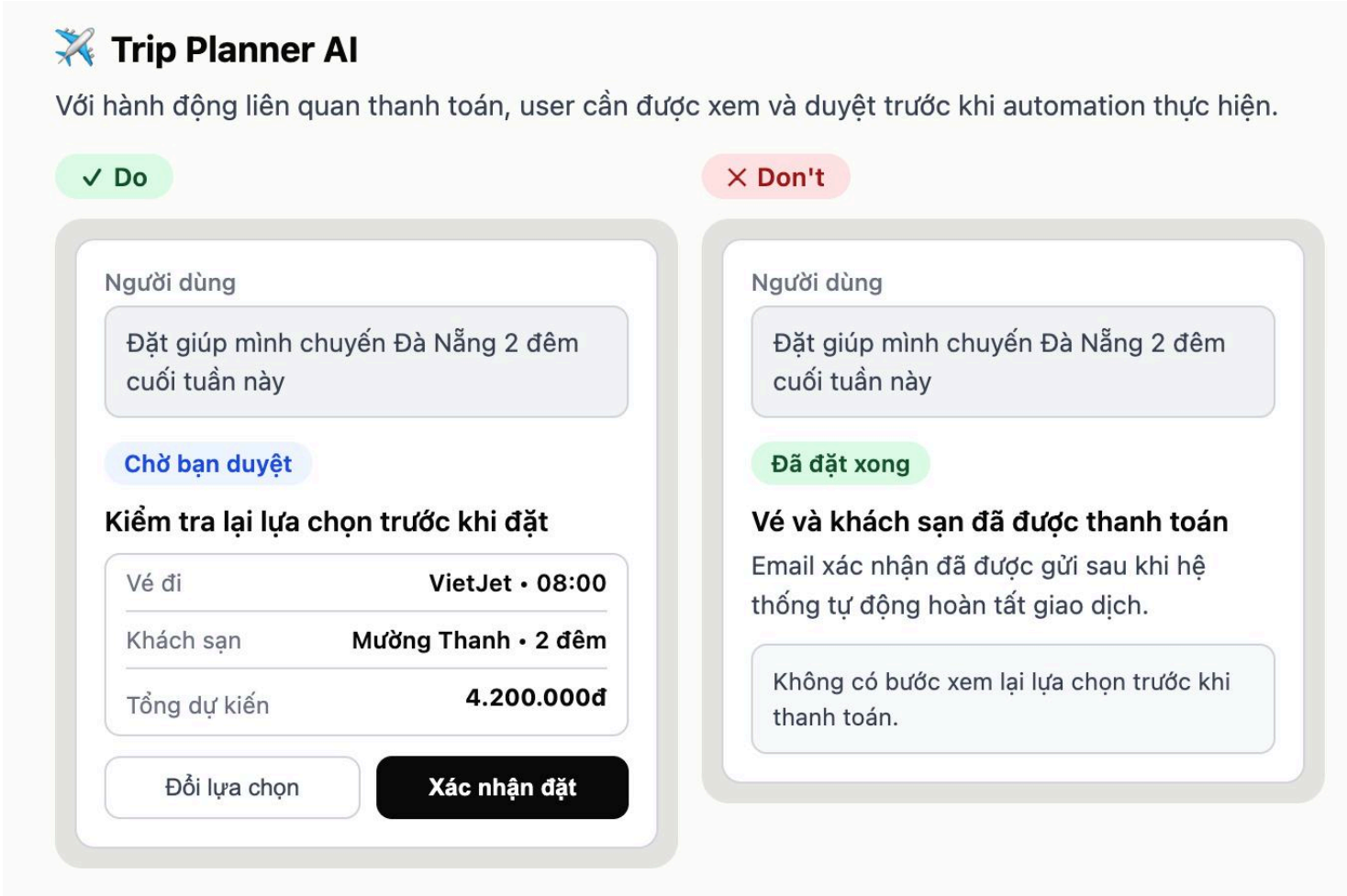
Map system input attributes to system outputs: yếu tố đầu vào nào ảnh hưởng mạnh

What-if explanations: cho user thử đổi đầu vào để xem kết quả đổi ra sao

Source: Make clear why the system did what it did

PAIR Guidebook, ch. "Explainability + Trust" — explanation là cách duy trì trust đúng mức.

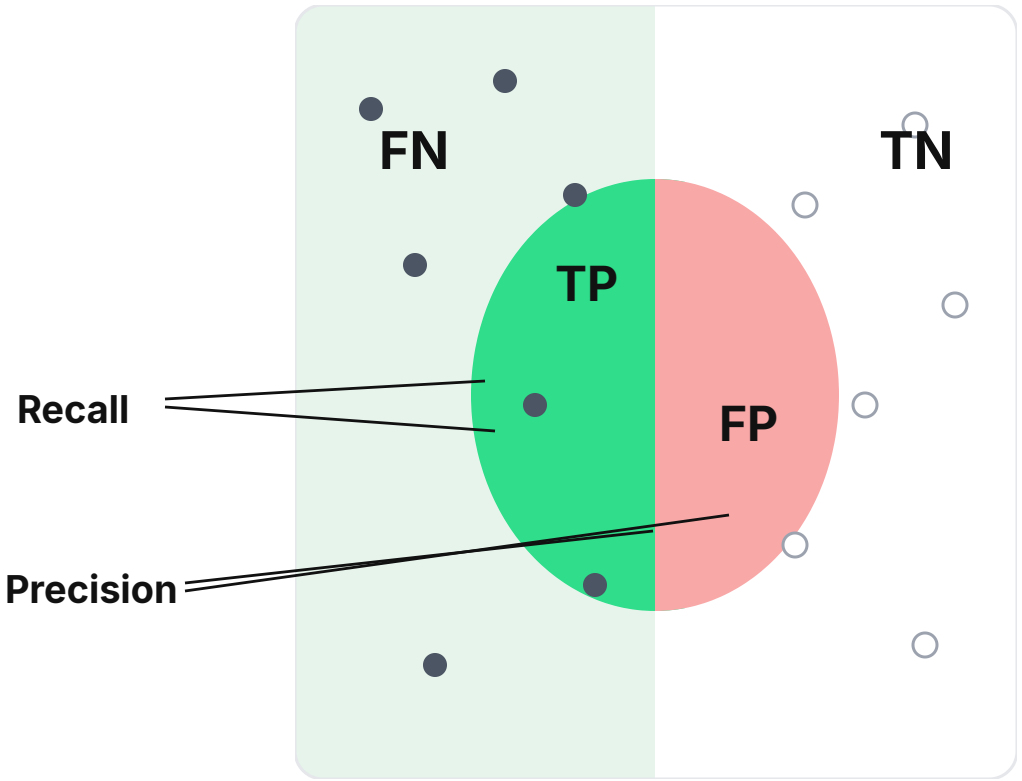
Cho phép user duyệt trước khi đi tiếp



AI có thể chuẩn bị. Người dùng mới là người phê duyệt. Friction thêm đó là friction tốt.

Precision / Recall tradeoff

*Chọn cách nào phụ thuộc vào việc **báo sai** hay **bỏ sót** gây hại hơn.*



Resume Screener

Trong tuyển dụng, nên giữ thêm các hồ sơ sát ngưỡng để HR tự rà, thay vì cắt cứng chỉ còn một nhóm rất nhỏ.

✓ Do

Người dùng

Sàng lọc 200 CV cho vị trí Product Designer

Danh sách nên cho HR rà tiếp

NT	Nguyễn Thảo		92%
LH	Lê Hùng		87%
TM	Trần Mai	Cần xem thêm	68%

Xem thêm 27 hồ sơ có thể phù hợp

LƯỢT RÀ TIẾP THEO

Những hồ sơ trái ngành, có career gap hoặc chưa khớp hoàn toàn vẫn còn cơ hội được HR xem thêm.

✕ Don't

Người dùng

Sàng lọc 200 CV cho vị trí Product Designer

Danh sách bị cắt quá hẹp

NT	Nguyễn Thảo		92%
LH	Lê Hùng		87%
TM	Trần Mai	Đã ẩn	68%

Chỉ hiển thị 5 hồ sơ đầu tiên

HỆ QUẢ

Những hồ sơ sát ngưỡng biến mất khỏi lượt rà đầu, nên HR không còn cơ hội xem thêm dù đó vẫn có thể là ứng viên phù hợp.

Hiển thị kết quả theo mức độ tự tin

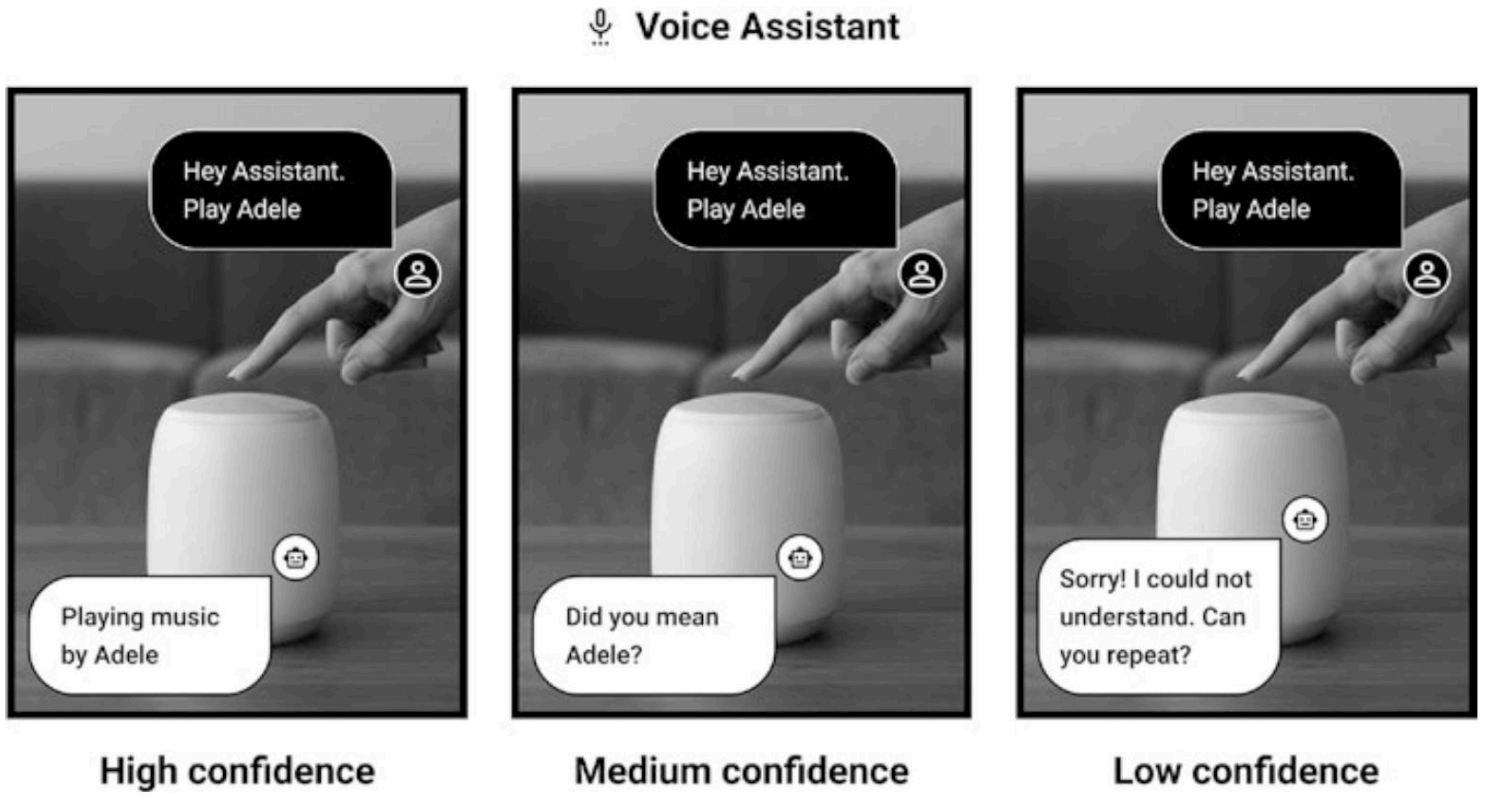


Figure 4-16. Categorical confidence. *Based on the confidence level, the voice assistant provides different responses*

User không cần biết 0.71 hay 0.84 — user cần thấy hệ thống cư xử khác nhau khi độ chắc khác nhau.

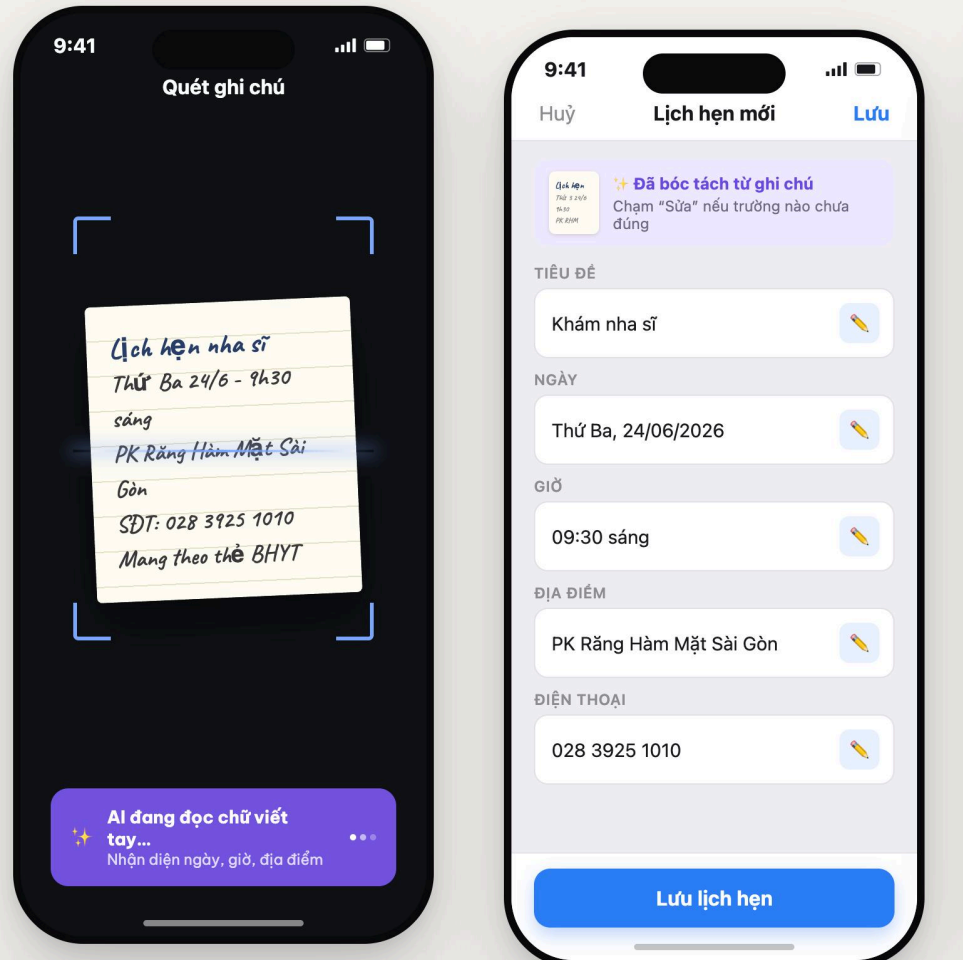
Kayak từng hiển thị "Confidence 79%" (2013–2019) — nay đã bỏ số %: confidence không nhất thiết là một con số (Kayak Help).

RECOVERY

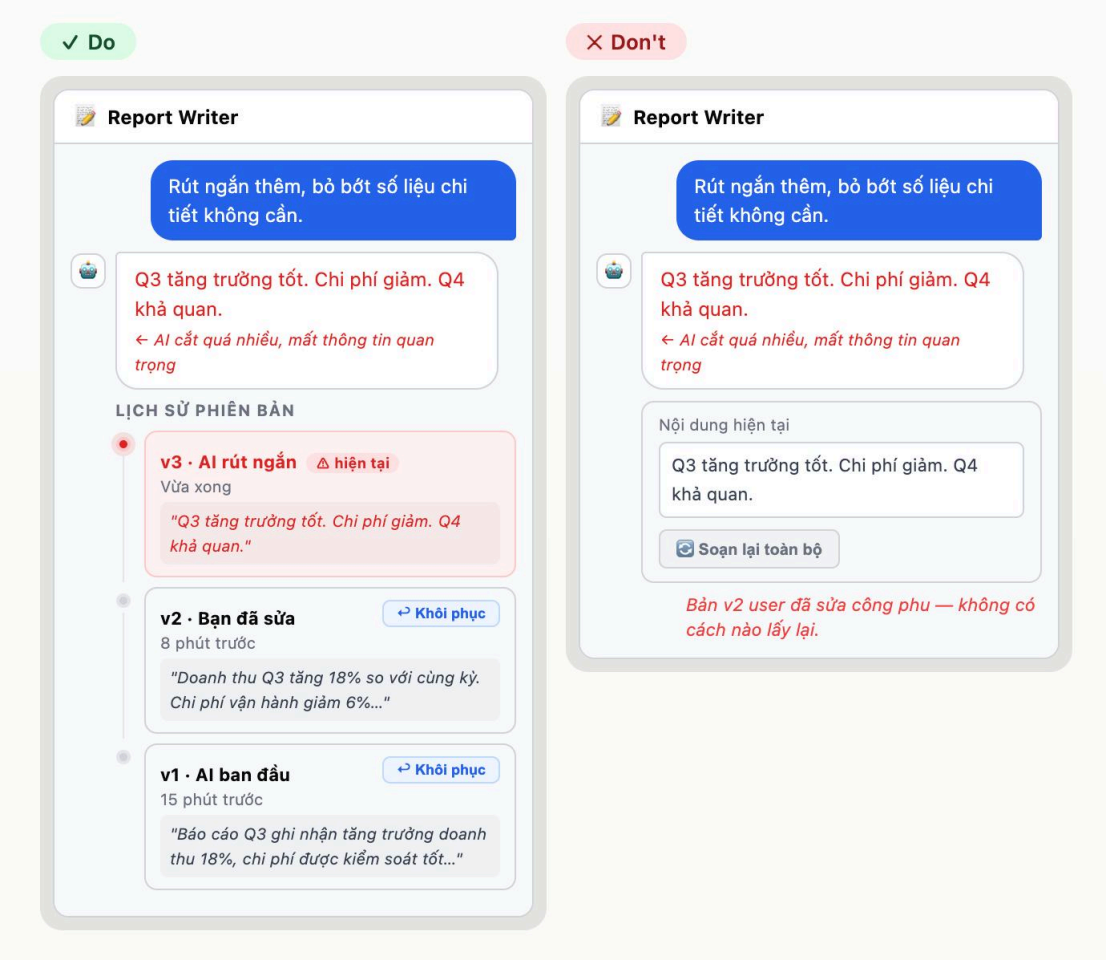
Thiết kế khi AI sai

Nửa đầu giảm khả năng sai. Nửa này: khi sai rồi, user không bị kẹt.

Cho phép user chỉnh sửa kết quả



Undo / Rollback



Nếu AI không hoàn hảo, đừng bắt user làm lại từ đầu.

Màn hình lỗi là cơ hội để hướng dẫn cách dùng đúng

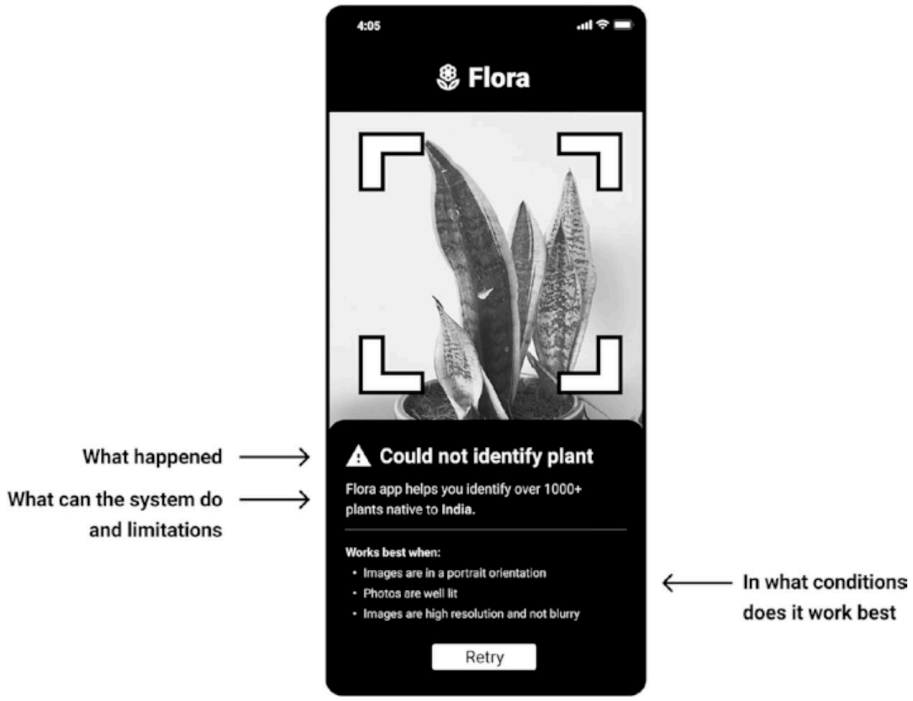


Figure 6-4. Use errors as opportunities for an explanation. *You can use errors to explain how the system works, its limits, and its capabilities, and they can help you set the right user expectations of your AI. In this example, the error message indicates that the system works on plants native to India and the conditions in which the system works best*

Lỗi là cơ hội để xin feedback

Figure 6-5. Use errors as opportunities for feedback. *The dog detector application misclassifies a muffin as a chihuahua in this example. Along with the prediction, the application allows the user to provide feedback. Image source: www.huffpost.com/entry/blueberry-muffin-chihuahua_n_4649064*

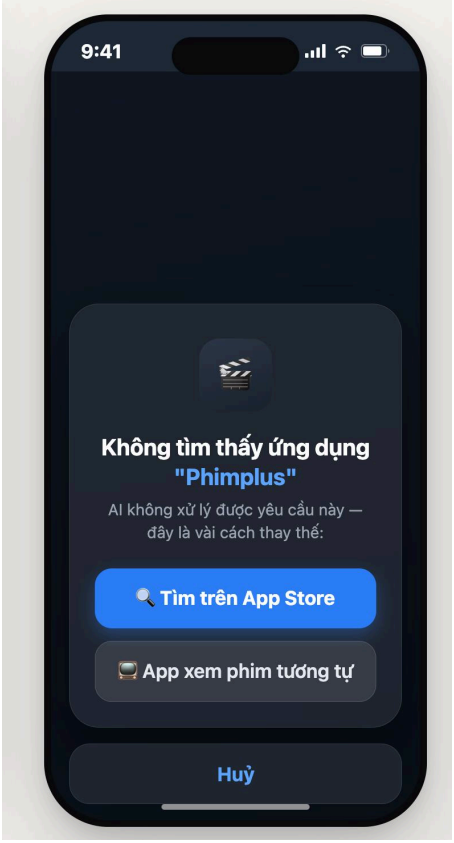
Error state là lúc user sẵn sàng học nhất — và cũng sẵn sàng phản hồi nhất.

Trả quyền kiểm soát cho người dùng

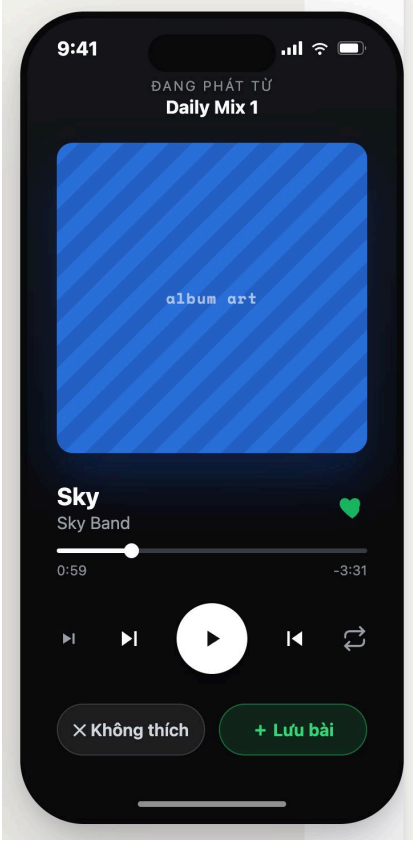
Cung cấp lối thoát rõ ràng khi AI không đủ khả năng

Chuyển sang người thật

Gợi ý bước tiếp theo



Có chế độ cho user tự chỉnh



AI tốt không phải AI luôn có câu trả lời — AI tốt biết đưa user sang con đường khác khi mình không đủ khả năng.

SECTION 04

Evals cơ bản

Sai kiểu nào tệ hơn · Precision hay Recall · Ba giai đoạn của eval flow — phần chuyên sâu gặp lại ở Ngày 6

Eval: sai kiểu nào tệ hơn?

Ví dụ: AI lọc video cho app trẻ em — 100 video, 10 video xấu thật

	XẤU THẬT (10)	LÀNH THẬT (90)
AI đánh dấu XẤU (13)	8 ✓ Chặn đúng	5 × Báo nhầm — video tốt bị gỡ oan
AI cho qua (87)	2 × BỎ SÓT — trẻ thấy nội dung xấu	85 ✓ Cho qua đúng

PRECISION

8 / 13 = 62%

Khi AI nói CÓ, đúng bao nhiêu?

RECALL

8 / 10 = 80%

Trong số cần tìm, AI tìm được bao nhiêu?

Cái nào tệ hơn? Lọt 2 video xấu (trẻ thấy) tệ hơn gỡ oan 5 video tốt → cần RECALL cao.

Precision = chặn đúng / tổng bị đánh dấu = 8/13 · Recall = chặn đúng / tổng xấu thật = 8/10.

Precision hay Recall — phụ thuộc context

	User act theo kết quả sai — FP tệ hơn → PRECISION	Bỏ lọt = mất giá trị — FN tệ hơn → RECALL
User THẤY & sửa được	**Legal RAG chatbot** User thấy câu trả lời, nhưng sai mà act theo → hậu quả pháp lý nặng	**Copilot, FAQ chatbot** Gợi ý nhiều, user tự lọc — bỏ lọt gợi ý hay = mất giá trị
User KHÔNG thấy	**Spam filter, auto-send email** Sai mà không ai biết = nguy hiểm (email quan trọng vào spam)	**Content mod trẻ em, fraud** Bỏ lọt = thảm họa — Recall bất kể user thấy hay không

Sai mà user KHÔNG BIẾT → thường cần Precision. Nhưng bỏ lọt = thảm họa → Recall bất kể user thấy hay không.

Precision hay Recall?

DISCORD — 5 PHÚT

Với mỗi sản phẩm: **sai kiểu nào tệ hơn** — báo nhầm (ưu Precision) hay bỏ lọt (ưu Recall)?

01
Lọc nội dung trẻ em
Bỏ lọt = trẻ xem được video xấu. Báo nhầm = video tốt bị gỡ oan.

02
Code autocomplete
Gợi ý sai user thấy ngay và gõ đè. Thiếu gợi ý = mất giá trị.

03
AI đọc X-quang
Bỏ sót khối u vs báo động giả cho bác sĩ kiểm tra thêm.

04
Duyệt khoản vay
Cho vay người không trả được vs từ chối nhầm khách tốt.

05
Gợi ý nhạc
Gợi ý dở thì user skip. Bỏ sót bài hay thì user không biết.

Gõ Discord 5 dòng: [số]-P hoặc [số]-R + lý do 1 câu · VD: 3-R — bỏ sót khối u nguy hiểm hơn báo nhầm

Không có đáp án tuyệt đối — chấm theo lý do thuyết phục.

Demo chạy tốt **không có nghĩa** sản phẩm chạy tốt

Vì sao phải đánh giá chất lượng AI (eval) thành một chu trình — không phải chấm điểm một lần.

Lúc demo

mọi thứ trong tầm kiểm soát

- 10–20 case do team **tự chọn**.
- Input "sạch", đúng kịch bản đã chuẩn bị.
- Chạy vài lần thấy ổn → kết luận "xong".

Lúc user thật dùng

không còn kiểm soát được input

- Hàng nghìn câu hỏi mỗi ngày — **không giống lúc demo**.
- User hỏi theo cách team chưa từng nghĩ tới.
- AI chắc chắn sẽ có lúc sai — vấn đề là **sai bao nhiêu, sai ở đâu**.

Chất lượng AI là một phân bố — đúng bao nhiêu %, trên loại case nào. Muốn biết con số đó thì phải đo, và phải đo liên tục.

Eval khác test phần mềm thường ở chỗ nào

Eval (đánh giá chất lượng AI) = chấm AI trên một **bộ case đại diện**, lặp đi lặp lại — trước và sau khi ra mắt.

	Test phần mềm thường	Eval cho AI
Kết quả	Cùng input → cùng output. Pass hoặc fail, rõ ràng.	Cùng input → mỗi lần một khác. "Đúng" là chuyện **mức độ**: đúng bao nhiêu %?
Bộ câu hỏi	Viết một lần, chạy mãi.	Phải **lớn dần theo case thật** từ user — không bao giờ "đủ".
Khi nào đo	Trước khi release (ra mắt).	Trước release **và** liên tục sau release — user tạo case mới mỗi ngày.

Precision / Recall hồi nãy chính là một kiểu thước đo của eval — chọn thước đo nào là một quyết định sản phẩm.

Ba giai đoạn của eval flow — nói trước bằng lời

Slide tiếp theo là hình tổng kết — nắm 3 khái niệm này trước đã.

01 · Vibe Check

chấm tay, cảm tính

- Chạy thử 10–30 case rồi **tự chấm tay**.
- Mục đích: hiểu AI hay **sai kiểu gì** — chưa cần con số chính thức.
- Khi nào: lúc còn prototype — trước cả khi viết PRD.

02 · Offline Eval

chấm tự động, trước ra mắt

- Có **bộ câu hỏi chuẩn** (reference dataset). Mỗi lần đổi prompt / model → chạy lại toàn bộ, so với phiên bản hiện tại.
- Qua "cổng chất lượng" (quality gate) mới được release.
- Cái từng chạy tốt nay tệ đi = **regression** (lỗi quay đầu).

03 · Online Monitoring

theo dõi sau ra mắt

- User thật tạo case mới **không lường trước**.
- Gom tín hiệu: thumbs up/down, user gõ lại prompt, bỏ giữa chừng.
- Case lạ → đưa ngược về bộ câu hỏi chuẩn.

Case thật từ online chảy ngược về bộ câu hỏi offline — bộ câu hỏi ngày càng chuẩn. Vì vậy gọi là chu trình, không phải chấm một lần.

Ba giai đoạn

STAGE 01

Vibe Check

Manual review để hình thành intuition trước khi đóng cứng spec

Prototype phase

STAGE 02

Offline Eval

So sánh, phát hiện regression, đặt quality gate trước rollout

Build phase

STAGE 03

Online Monitoring

Theo dõi sau launch, bắt drift và failure mode mới

Production phase